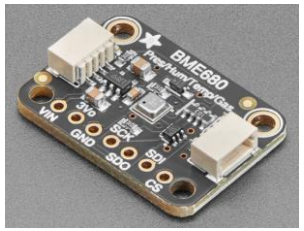
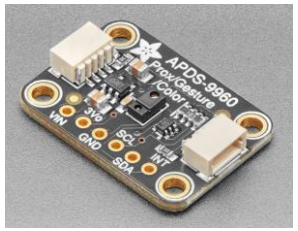
Academic Program Workshop: Microchip PyKit Explorer

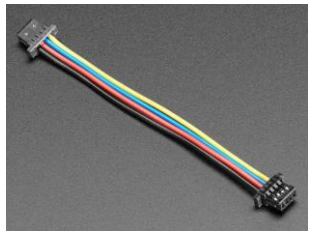
A Leading Provider of Smart, Connected and Secure Embedded Control Solutions

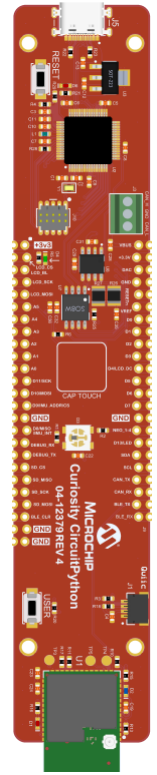
What's In The Kit?

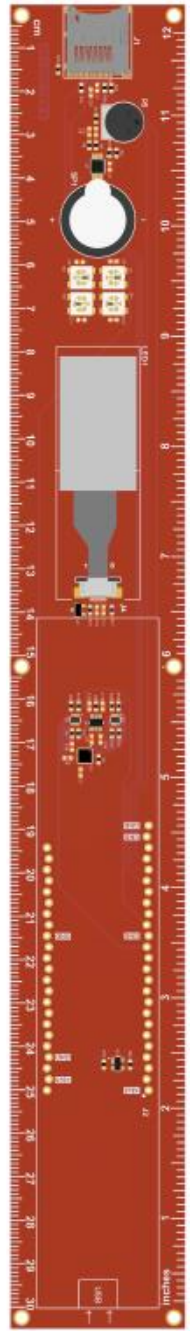
- **Curiosity CircuitPython dev board**
- **PyKit Ruler**
- **Adafruit BME680 Breakout:**
  - Temp, Humidity, Pressure, Gas sensor
- **Adafruit APDS9960 Breakout:**
  - Proximity, Gesture, Color sensor
- **QWIIC / STEMMA cable**
- **USB-A to USB-C cable**
- **Header Pins to connect Ruler and dev board**
  - 1 x 23-pin, 1 x 25-pin





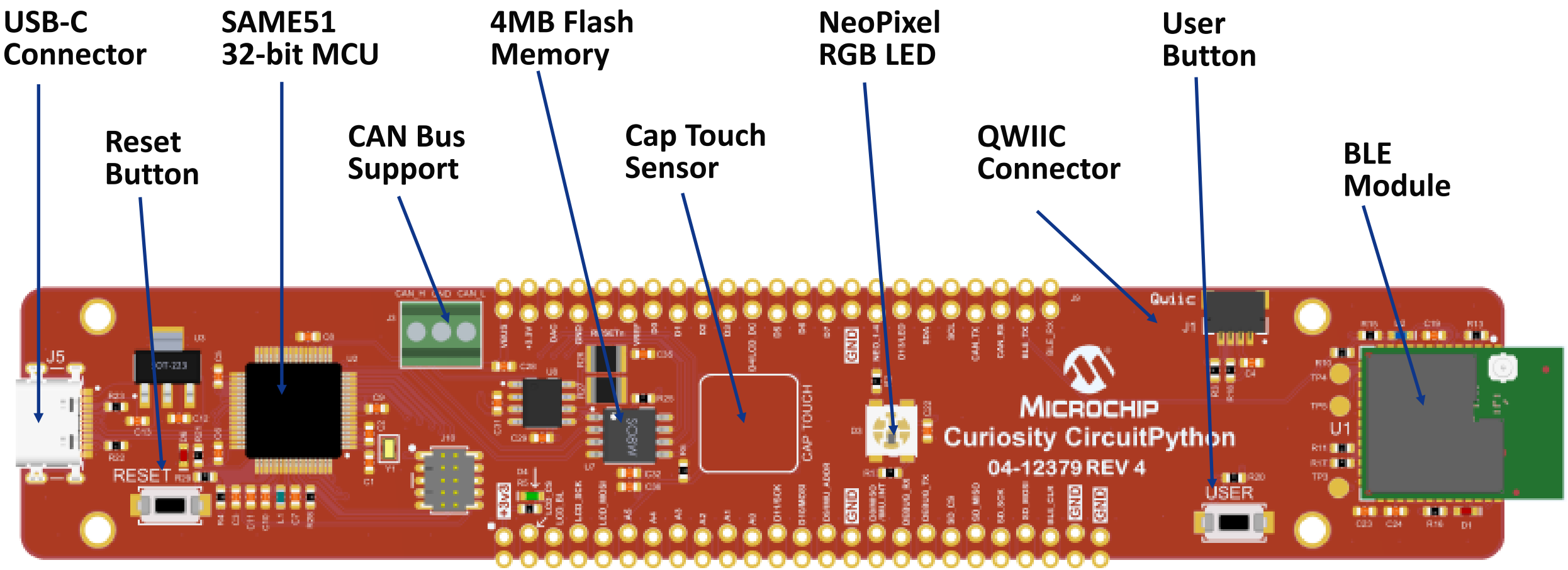






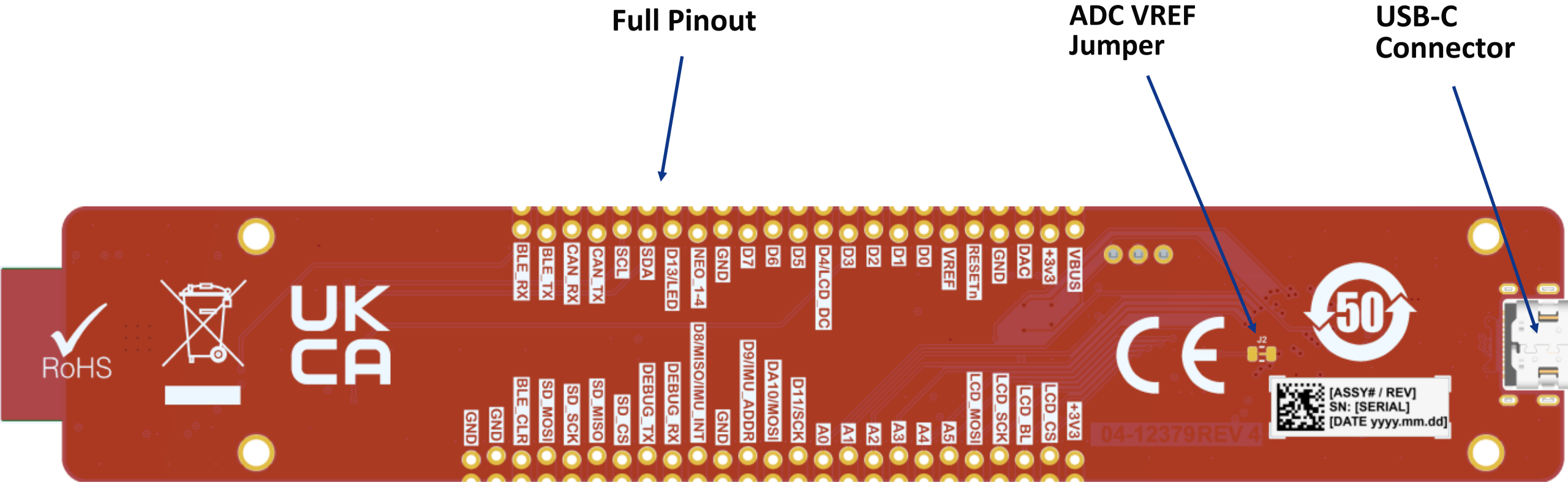
What Hardware Does The Kit Have?

Curiosity CircuitPython dev board



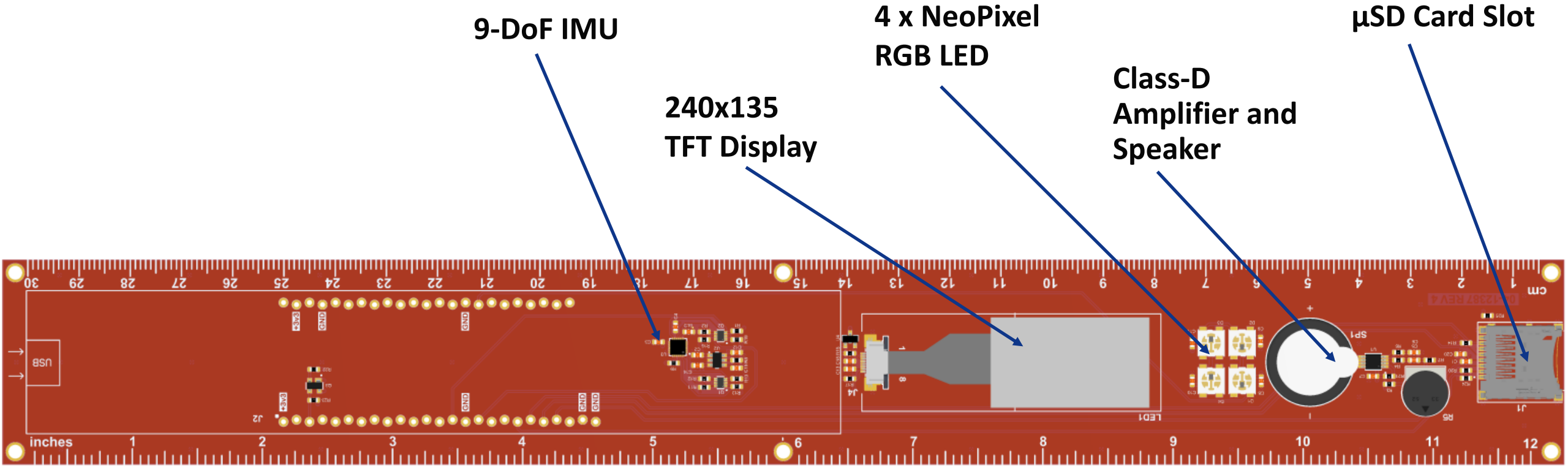
What Hardware Does The Kit Have?

Curiosity CircuitPython dev board



What Hardware Does The Kit Have?

PyKit Ruler baseboard



What Hardware Does The Kit Have?

PyKit Ruler baseboard - backside

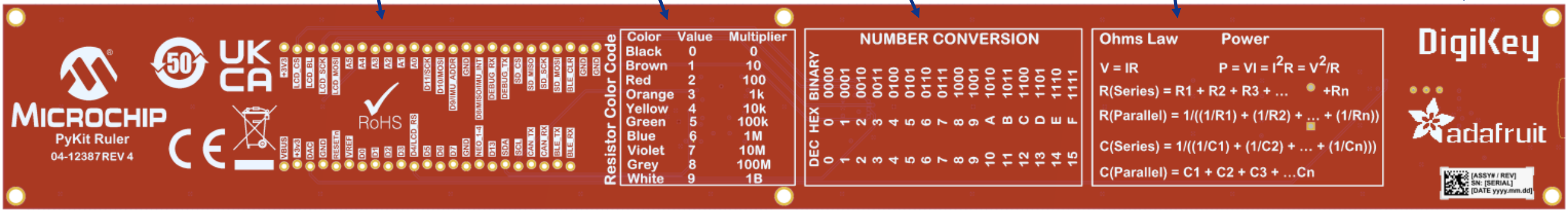
Full Pinout

Resistor Color Code

Decimal
Binary
Hex
Number Conversions

Useful Equations:
Ohm's Law
Series/Parallel Resistors/Capacitors

Our Partners

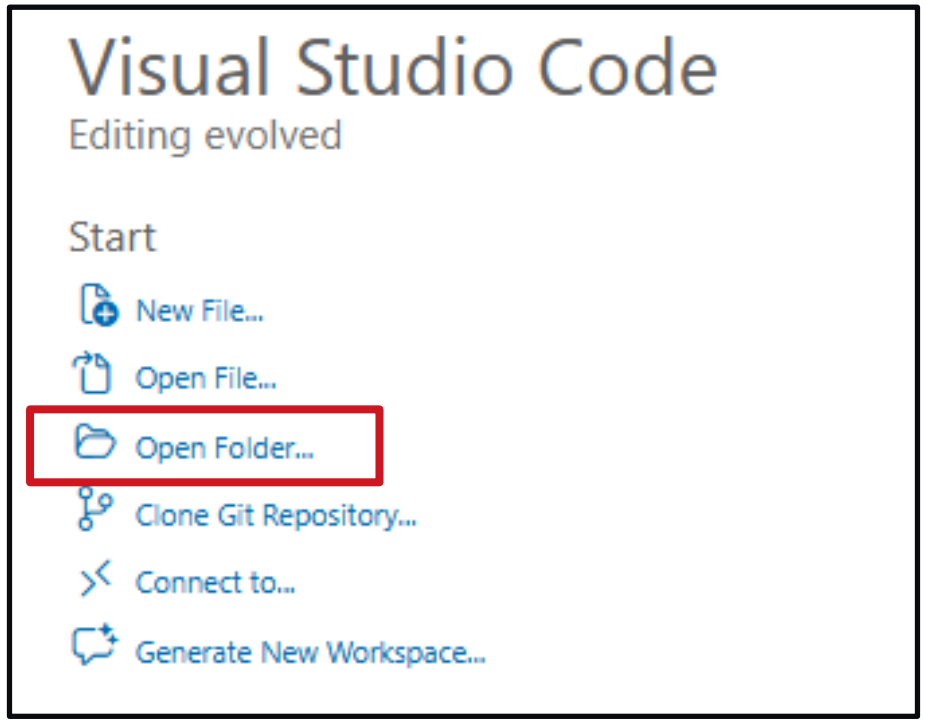


MICROCHIP

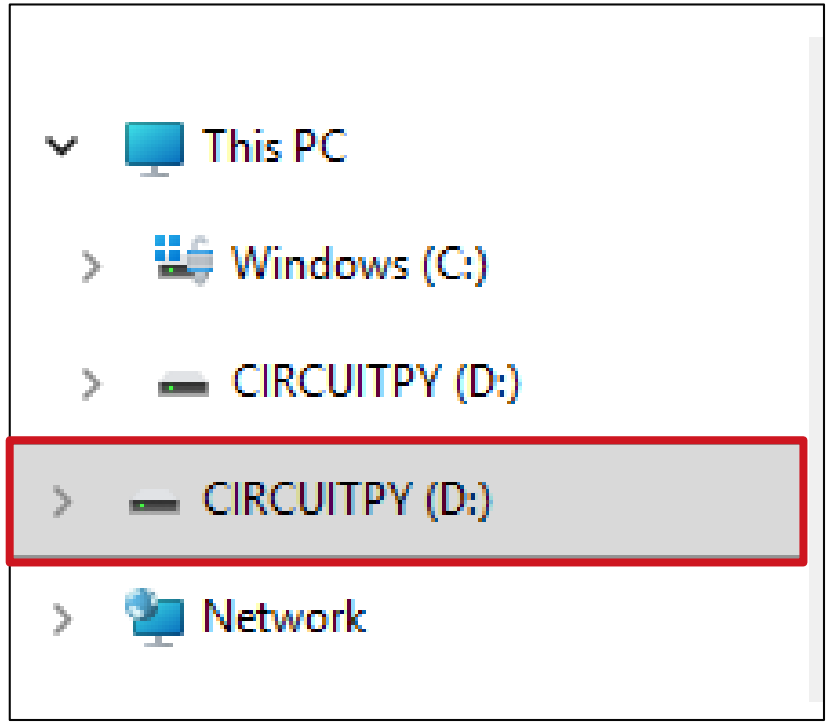
Editing CircuitPython Code in VS Code

Connect CircuitPython kit to your computer with USB cable

1. Open VS Code:



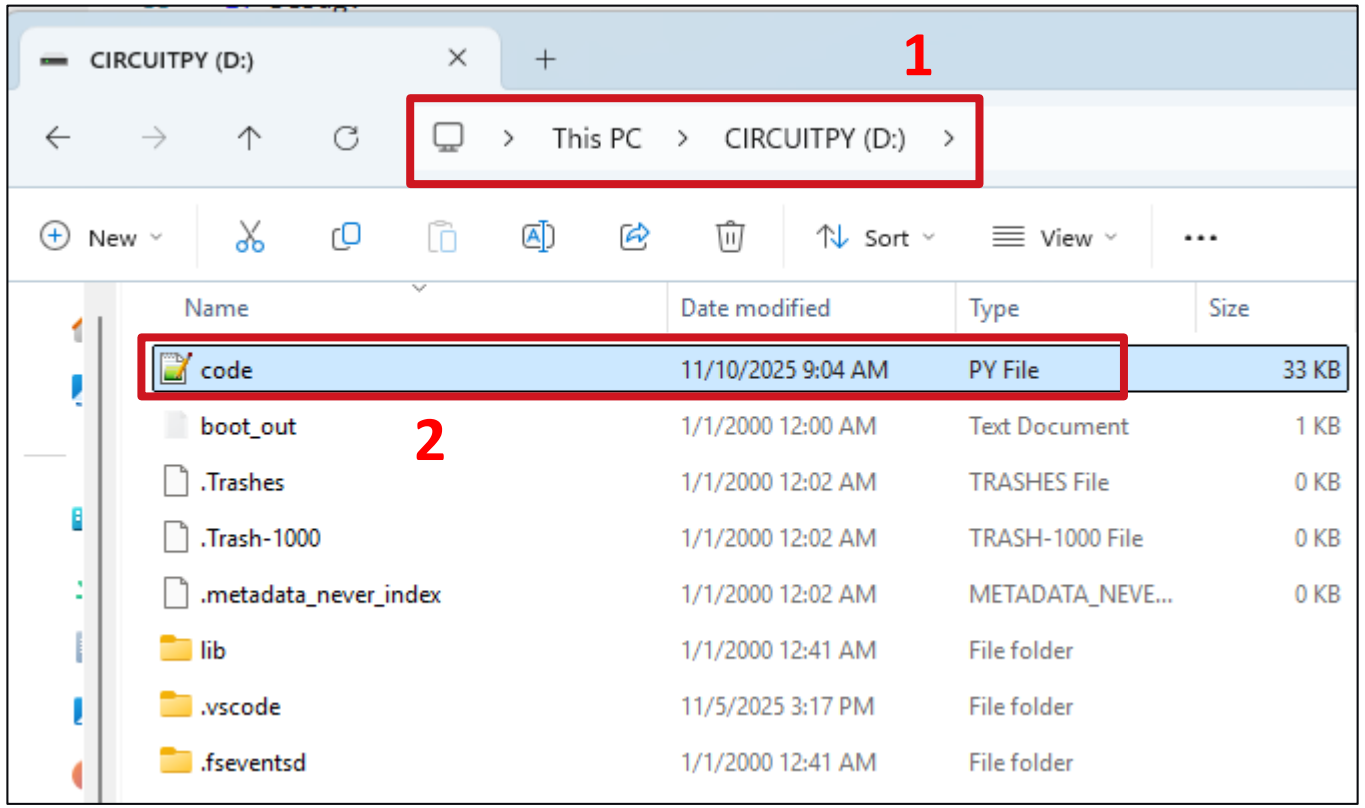
2. Open CIRCUITPY folder:



MICROCHIP

How to Run Your Code

code.py file



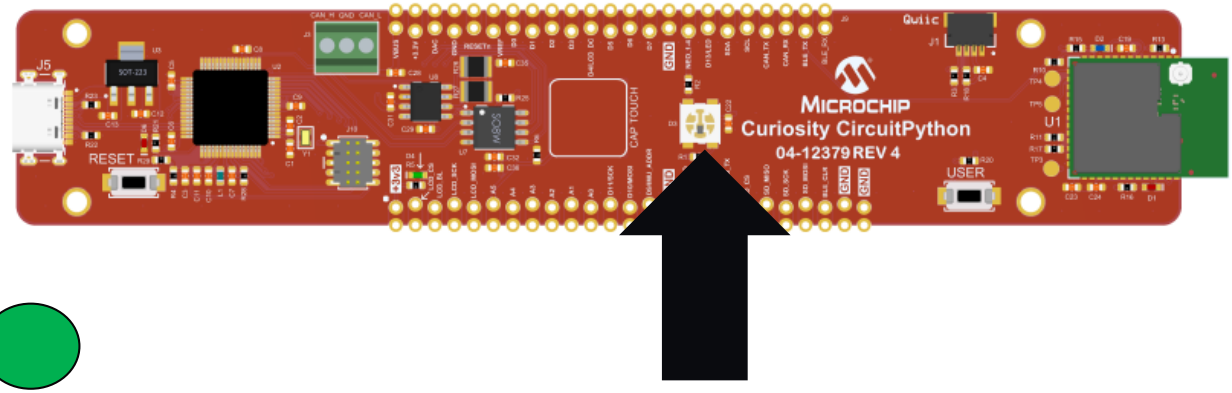
For your code to run on the board, it **must**:

1. Be named ***code.py***
2. Be in the **root directory** of D:\CIRCUITPY

MICROCHIP

NeoPixel Status Indicator

Visual Error Indication



- **NeoPixel - 1 Green Flash**
  - Code completed successfully
- **NeoPixel - 2 Red Flashes**
  - Code encountered an error – Check the Serial Monitor for info
- **NeoPixel - 3 Yellow Flashes**
  - Safe Mode entered after bad crash!
  - Need to preset Reset button to exit Safe Mode

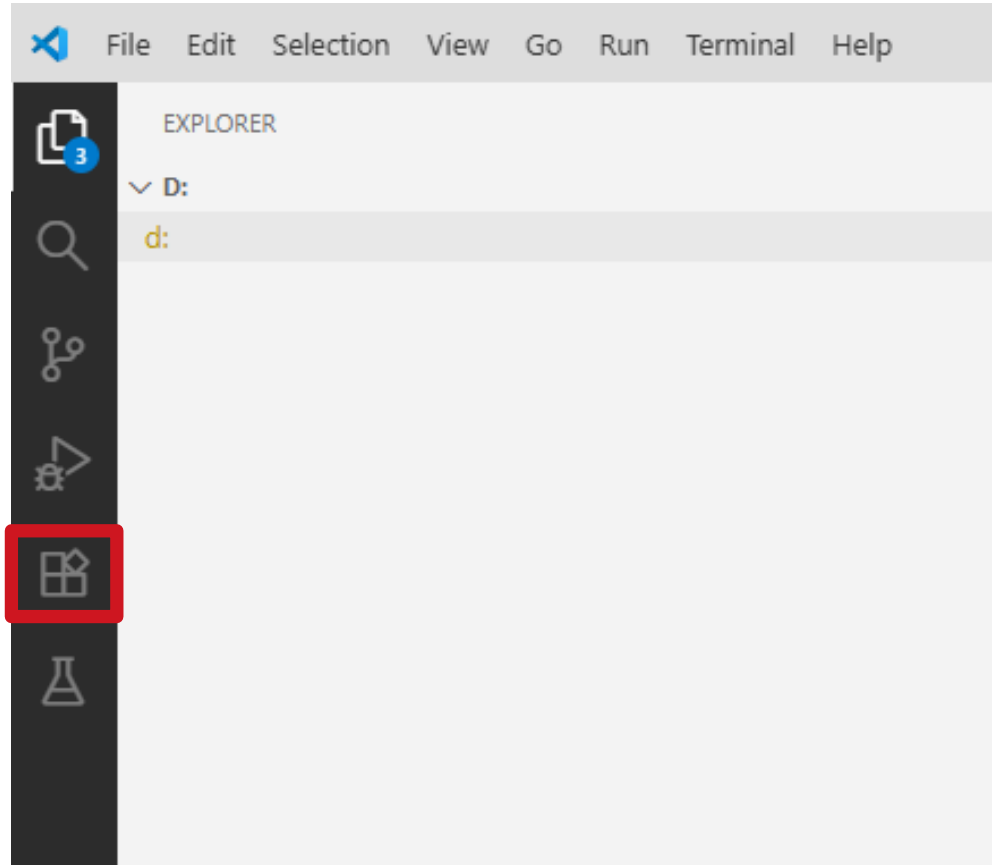
- **Note:** Do NOT confuse NeoPixel flash with BLE module **Blue LED** (advertising mode) flash!

MICROCHIP

Open Serial Monitor

VS Code Extension

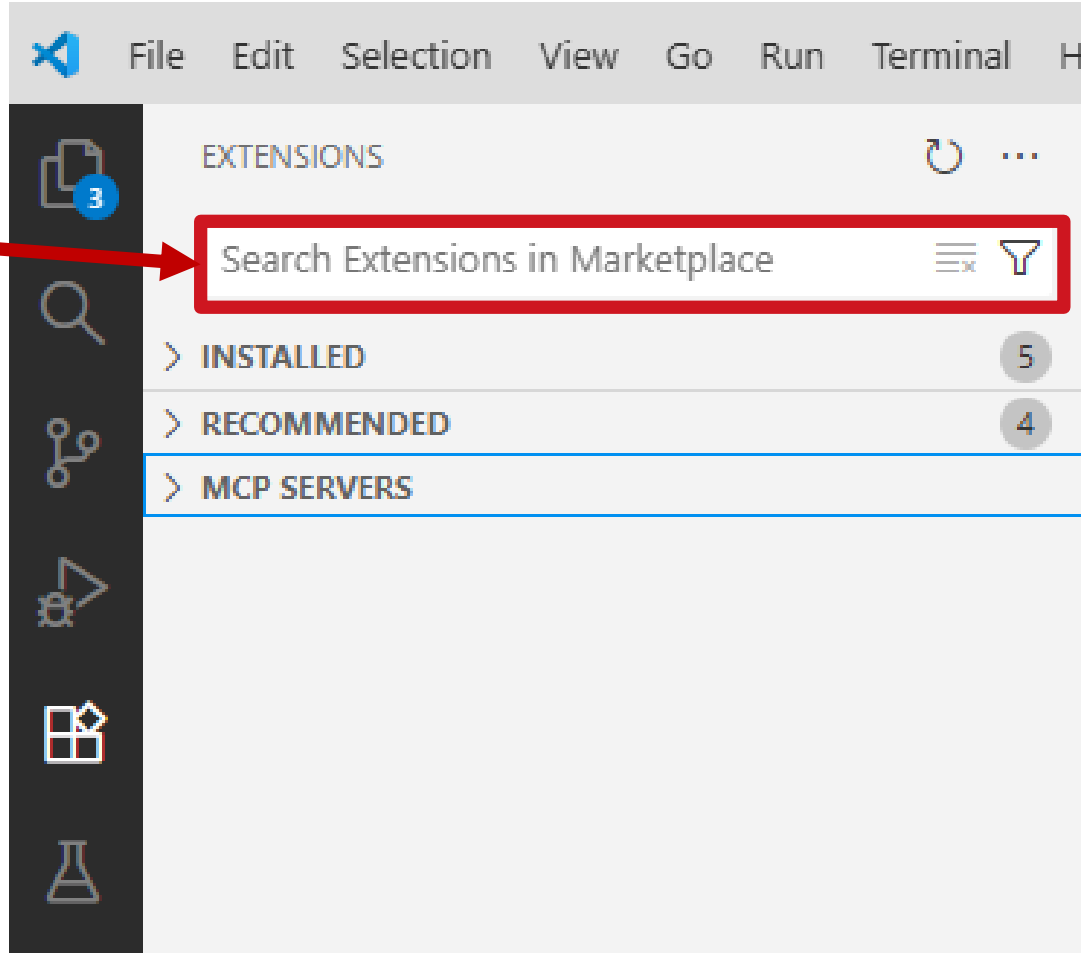
- Open *VS Code*
- Click on *Extensions* icon on left side



Search For Serial Monitor Extension

VS Code Extensions

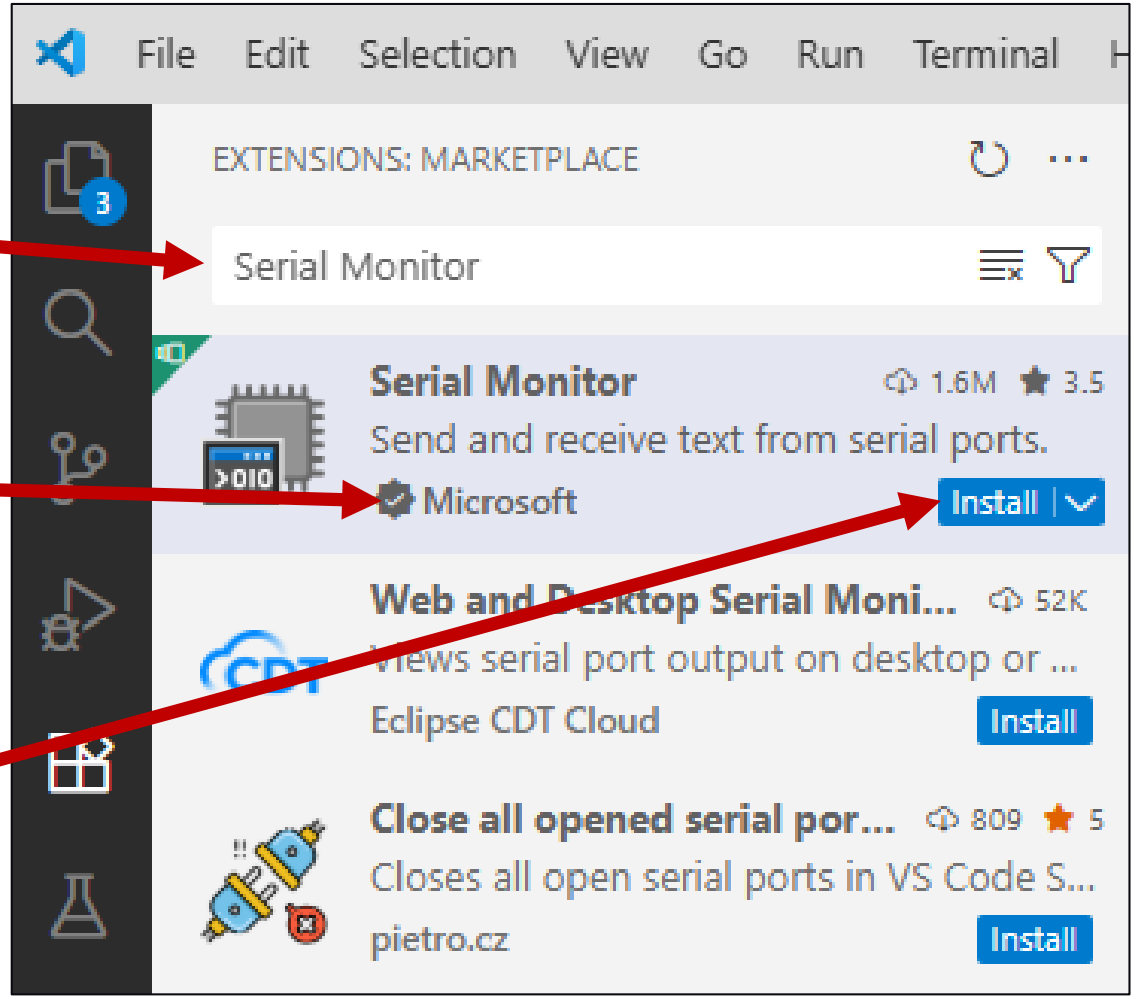
- **Click on text field:**
  - *Search Extensions in Marketplace*



Enter “Serial Monitor” in Text Field

VS Code Extensions

- **Type in Text Field:**
  - *Serial Monitor*
- **Confirm author is *Microsoft***
- **Click *Install* button**

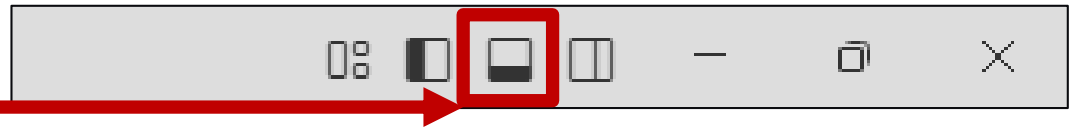


MICROCHIP

Open Serial Monitor

VS Code - Toggle Panel

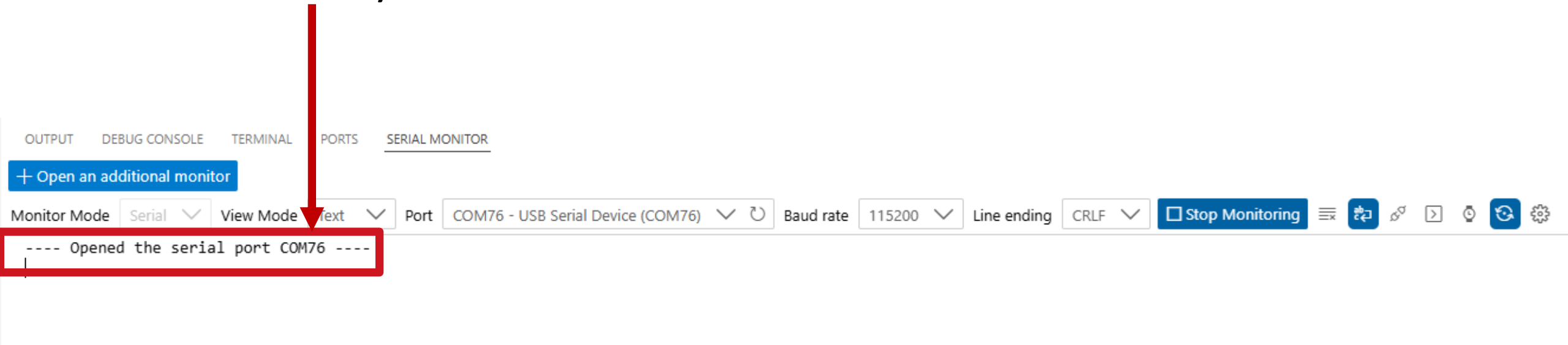
- **In the top right corner**
  - Click *Toggle Panel*



1. **Select your COM port:**
   - *USB Serial Device*
2. **Line Ending:**
   - *CRLF*
3. **Click *Start Monitoring***

Serial Monitor Opened

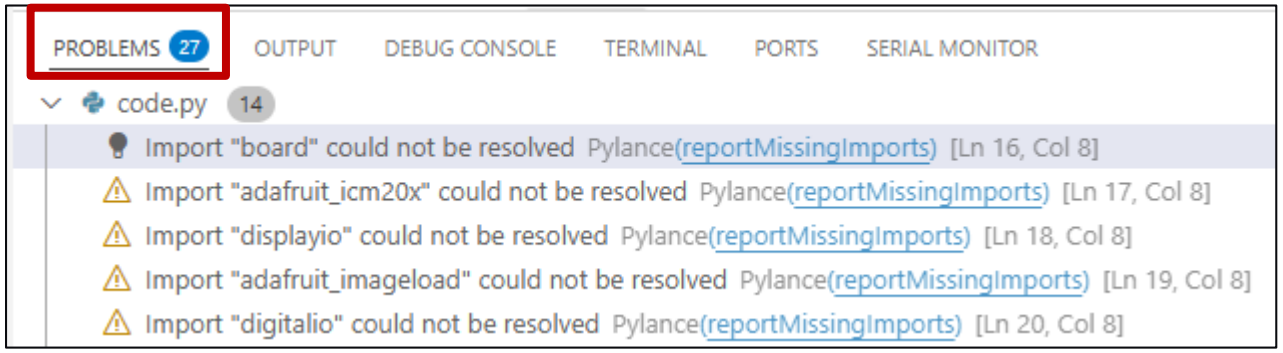
- **You will see:**
  - *---- Opened the Serial Port COMxx ----*
  - Where: *xx* is your Serial Port Number

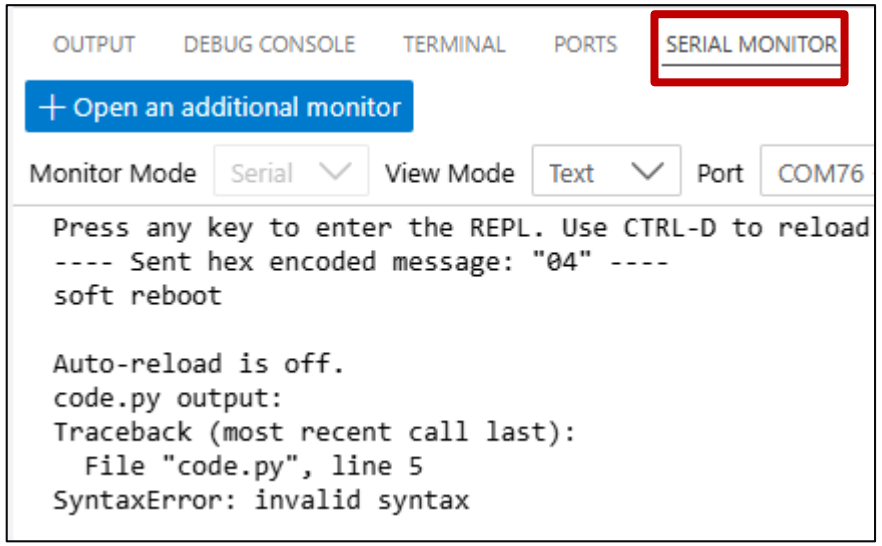


IMPORTANT NOTE ABOUT ERRORS!

Use “Serial Monitor” – Not “Problems”

- The extension ***CircuitPython v2*** does **NOT YET** support this board.
- ***Problems*** tab shows “problems” due to this extension not yet supporting this board
- Use ***Serial Monitor*** tab to view code errors





Getting Started

The README has what you need!

Check the README

Directory Layout

Directory Layout

All modules live flat in `/API` on the CIRCUITPY drive (Adafruit libraries stay in `/lib`):

```
CIRCUITPY/
├── code.py
├── lib/                        ← Adafruit / third-party libraries
│   ├── asyncio/
│   ├── adafruit_st7789.mpy
│   └── ...
└── API/                        ← PyKit Ruler modules
    ├── digital_io.py           ← Dev board modules
    ├── analog_io.py
    ├── pwm_out.py
    ├── cap_touch.py
    ├── servo_control.py
    ├── uart_comms.py
    ├── i2c_bus.py
    ├── spi_bus.py
    ├── hid_input.py
    ├── cpu_temp.py
    ├── ble_uart.py
    ├── can_bus.py
    ├── neopixels.py            ← Ruler baseboard modules
    ├── lcd_display.py
    ├── imu_sensor.py
    ├── audio_out.py
    ├── sd_card.py
    ├── bme680.py               ← I2C breakout modules (QWIIC)
    ├── apds9960.py
    └── async_tasks.py          ← Utility modules
```


MICROCHIP

Getting Started – Before Building a Project

The README has what you need!

Check the README Module Quick Reference

Module Quick Reference

Dev Board Modules

Module	Class(es)	What it does
`digital_io`	`DigitalOutput`, `DigitalInput`, `EdgeDetector`	Read buttons/switches; drive LEDs and relays; detect press/release edges
`analog_io`	`AnalogInput`, `AnalogOutput`	Read voltages from sensors (A0–A5); output DC voltage from DAC (board.DAC only)
`pwm_out`	`PWMOutput`	Variable duty-cycle signal; LED dimming; buzzer tones; motor speed control
`cap_touch`	`CapTouch`	Capacitive touch detect/release on board.A5 (CAP1)
`servo_control`	`ServoController`	Position standard RC servo 0°–180°; sweep animations
`uart_comms`	`UARTComms`	Send/receive strings over hardware UART (DEBUG or any UART)
`i2c_bus`	`I2CBus`	Scan I2C bus; raw register reads/writes; returns bus object for Adafruit drivers
`hid_input`	`HIDKeyboard`, `HIDMouse`, `JoystickMouse`	USB HID keyboard typing and key combos; mouse movement and clicks; joystick → mouse
`cpu_temp`	`CPUTemperature`	On-chip temperature in °C and °F; threshold checks; formatted logging strings
`ble_uart`	`BLEUart`	Reset RNBD451 BLE module; send/receive strings wirelessly; connection status tracking
`spi_bus`	`SPIBus`	General-purpose SPI transactions with automatic CS and bus locking
`can_bus`	`CANBus`	Send and receive CAN frames at 250 kbps; bus state monitoring

MICROCHIP

Getting Started – Before Building a Project

The README has what you need!

What Inputs and Outputs does your project need?

Choosing Modules for Your Project

Think through what your project needs to **sense**, **process**, and **output**:

INPUTS		OUTPUTS	
digital_io	← buttons	digital_io	→ LEDs, relays
analog_io	← sensors	pwm_out	→ motors, buzzers
cap_touch	← touch pad	servo_control	→ servo position
imu_sensor	← motion/tilt	neopixels	→ RGB feedback
bme680	← temp/humidity	lcd_display	→ graphics
apds9960	← proximity	audio_out	→ sound / music
apds9960	← gesture	ble_uart	→ wireless data
apds9960	← color	sd_card	→ data logging
i2c_bus	← I2C devices	hid_input	→ PC automation
spi_bus	← SPI devices		
uart_comms	← serial devices		
can_bus	← CAN network		

MICROCHIP

Getting Started – Minimal Examples

The README has what you need!

The README has Minimal examples to get up and running quickly!

Minimal Example — Blink the onboard LED

```
1   import pykit_explorer
2   from digital_io import DigitalOutput
3
4   led = DigitalOutput(board.LED)
5
6   while True:
7       led.on()
8       time.sleep(0.5)
9       led.off()
10      time.sleep(0.5)
```


MICROCHIP

What's Actually Happening Here?

Is it Magic?

- **Adafruit Libraries are in CIRCUITPY/lib folder**
- **APIs are in CIRCUITPY/API folder**
- **The APIs are just a wrapper for Adafruit Libraries**
  - Simplifies building applications!
- **Import pykit_explorer**
  - Pulls in basic libraries and API

What's Actually Happening Here?

No Magic, Just Code!

```
1   import pykit_explorer
2   from digital_io import DigitalOutput
3
4   led = DigitalOutput(board.LED)
5
6   while True:
7       led.on()
8       time.sleep(0.5)
9       led.off()
10      time.sleep(0.5)
```

`import pykit_explorer`

- Is a hardware initialization shim
- Imports core CircuitPython modules (board, time, built-ins)
- Adds /API to the module search path
- Injects time and board into built-ins so all subsequent scripts can use them without explicit imports

MICROCHIP

What's Actually Happening Here?

No Magic, Just Code!

```
1   import pykit_explorer
2   from digital_io import DigitalOutput
3
4   led = DigitalOutput(board.LED)
5
6   while True:
7       led.on()
8       time.sleep(0.5)
9       led.off()
10      time.sleep(0.5)
```

```
from digital_io import
DigitalOutput
```

Everyone open the **digital_io.py** file in CIRCUITPY/API

MICROCHIP

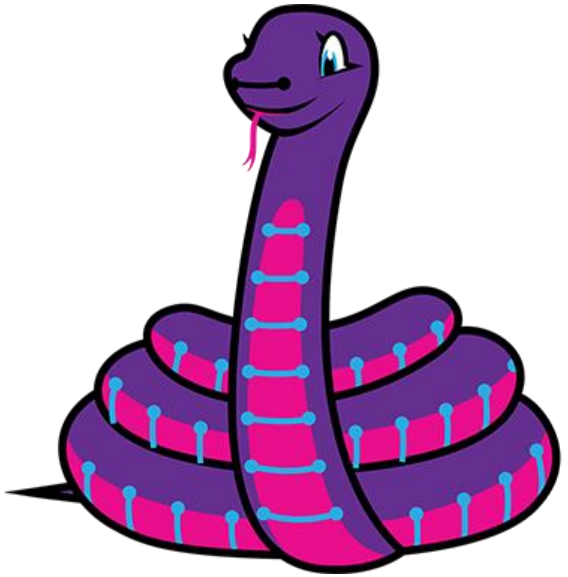
API > digital_io.py > ...

```
"""
digital_io.py — Digital Input / Output
=======================================
Board: Dev Board (any digital-capable pin)

Provides simple wrappers for reading digital inputs and driving digital outputs.
Use this module any time your project needs to:
  - Read a button, switch, or logic-level signal
  - Drive an LED, relay, transistor, or other digital load
  - Detect edges (press / release events)

Typical pins: D0–D7, D13 (LED), A0–A5 (also usable as digital I/O)
"""
```


MICROCHIP

What Methods Are Available to Us?

What can we do with this?



- **After the constructor we have:**
  - **`value`** – overloaded, can be getter or setter
  - **`On, Off, Toggle`**
  - **`Blink`** - On time, Off time, Count
- **The constructor points to the underlying CircuitPython library *digitalio***

```
def __init__(self, pin):
    self._pin = digitalio.DigitalInOut(pin)
    self._pin.direction = digitalio.Direction.OUTPUT
    self._state = False
```

Project 1

Blink the Onboard LED

Copy + Paste Examples: Blink the Onboard LED

README: Minimal Example #1

- **This is the embedded systems 'Hello World'**
  Every embedded systems learner starts here

- **import pykit_explorer**
  - Pulls in libraries: time, board, sys, builtins
  - Allows use of modules in /API folder

- **board.LED is the red LED near the USB port**
  Not the NeoPixel!

- **while True: runs forever**
  Without it, the code runs once and stops
  NeoPixel flashes GREEN once when code finishes

- **Try: change the sleep values - what happens if both are 0.1? What if on=0.1, off=1.0?**

Minimal Example — Blink the onboard LED

```
1   import pykit_explorer
2   from digital_io import DigitalOutput
3
4   led = DigitalOutput(board.LED)
5
6   while True:
7       led.on()
8       time.sleep(0.5)
9       led.off()
10      time.sleep(0.5)
```


MICROCHIP

Copy + Paste Examples: Blink the Onboard LED

How to find the modules required?

- **Open README.md**
- **Scroll down to:**
  - "Module Quick Reference"

Module Quick Reference

Dev Board Modules

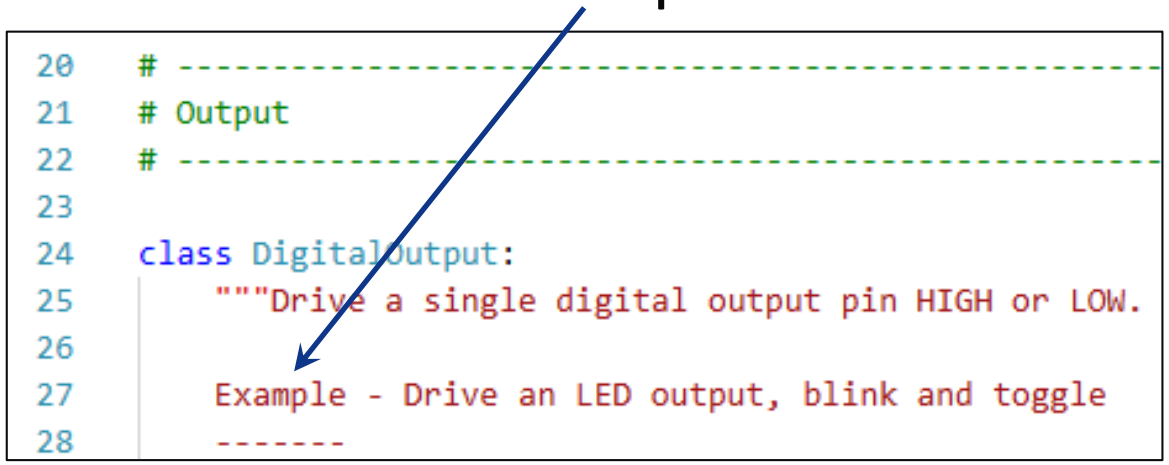
Module	Class(es)	What it does
`digital_io`	`DigitalOutput`, `DigitalInput`, `EdgeDetector`	Read buttons/switches; drive LEDs and relays; detect press/release edges

  - We want to drive LEDs, so we need module **digital_io**
  - Are LEDs inputs or outputs?
    - Which Class(es) should we use?

Copy + Paste Examples: Blink the Onboard LED

How to find the modules required?

- **Let's open the module "*digital_io*" in API folder**
- **Since our LED is an OUTPUT, we want Class DigitalOutput**
  - There's even an example!



```
20  # ------------------------------------------------------
21  # Output
22  # ------------------------------------------------------
23
24  class DigitalOutput:
25      """Drive a single digital output pin HIGH or LOW.
26
27      Example - Drive an LED output, blink and toggle
28      -------
```

- **Which methods are available to this Class?**

Copy + Paste Examples: Blink the Onboard LED

Methods available to Digital Output Class?

- **Constructor**

```
def __init__(self, pin):
    self._pin = digitalio.DigitalInOut(pin)
    self._pin.direction = digitalio.Direction.OUTPUT
    self._state = False
```

- **Value (getter)**

```
@property
def value(self):
    return self._pin.value
```

- **On**

```
def on(self):
    """Drive the pin HIGH."""
    self.value = True
```

- **Off**

```
def off(self):
    """Drive the pin LOW."""
    self.value = False
```

- **Toggle**

```
def toggle(self):
    """Flip the current output state."""
    self.value = not self._state
```

- **Blink**

```
def blink(self, on_time: float = 0.5, off_time: float = 0.5, count: int = 1):
    """Blink the output *count* times (blocking).
```

- **Deinit**

```
def deinit(self):
    self._pin.deinit()
```


MICROCHIP

Copy + Paste Examples: DigitalOutput API

The API has three ways to control an LED (or other output)

- **All three approaches produce a blinking LED**
  Each teaches something different:

- **on()/off() - explicit state control**
  Good for understanding what the hardware is doing at each line of code

- **toggle() - flip whatever state it is in**
  Useful when you don't know current state
  Fewer lines, same result

- **blink(on_time, off_time, count)**
  Blocking - code waits until done
  Great for status flashes

```
# Manual on/off with sleep
led.on()
time.sleep(0.5)
led.off()
time.sleep(0.5)
```

1

```
# Toggle flips the current state
while True:
    led.toggle()
    time.sleep(0.5)
```

2

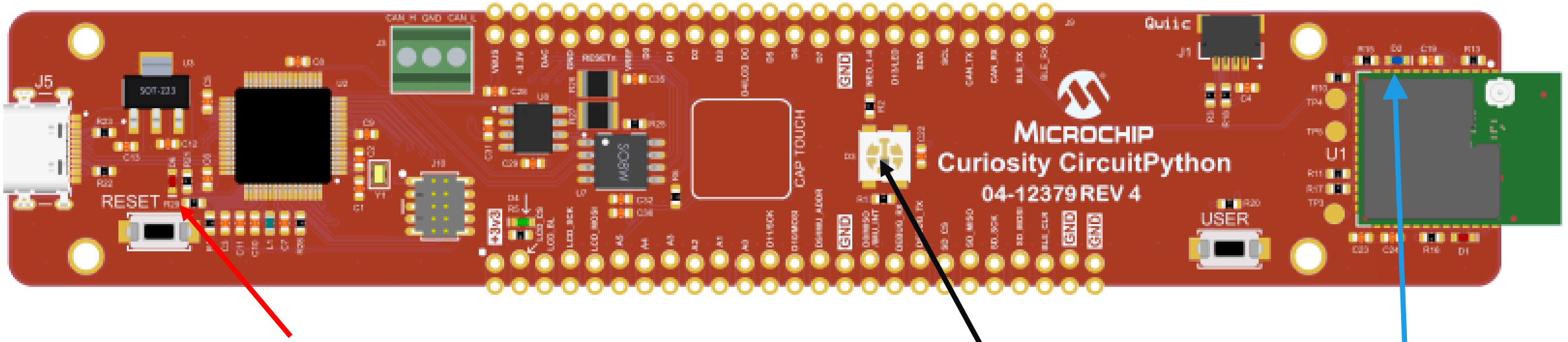
```
# blink() does it all in one call
led.blink(on_time=0.2,
          off_time=0.8,
          count=5)
```

3

MICROCHIP

Where Is The LED?

I can’t find it!



- There is a Red LED near the USB connector
- Do not confuse the LED and NeoPixel. The NeoPixel is next to the Cap Touch pad
- The Bluetooth LE module has a Blue LED that blinks periodically (Advertising Mode)

Project 2

Read the User Button

Copy + Paste Examples: Read User Button

README: Minimal Example #2

- **import pykit_explorer**
  - Pulls in libraries: time, board, sys, builtins
  - Allows use of modules in /API folder

- **The User button is connected to pin D3**
  - Refer to schematics

- **while True: runs forever**

  Without it, the code runs once and stops

  NeoPixel flashes GREEN once when code finishes

- **Prints *value* and *is_pressed***
  - What's the difference?

Minimal Example #2 — Read User Button

```
1  import pykit_explorer
2  from digital_io import DigitalInput
3  btn = DigitalInput(board.D3)
4
5  while True:
6      print(btn.value)          # True = not pressed (active-low)
7      print(btn.is_pressed())   # True when button pulled LOW
8      time.sleep(0.1)             # debounce delay
```


MICROCHIP

Copy + Paste Examples: Read User Button

How to find the modules required?

- **Open README.md**
- **Scroll down to:**
  - “Module Quick Reference”

Module Quick Reference

Dev Board Modules

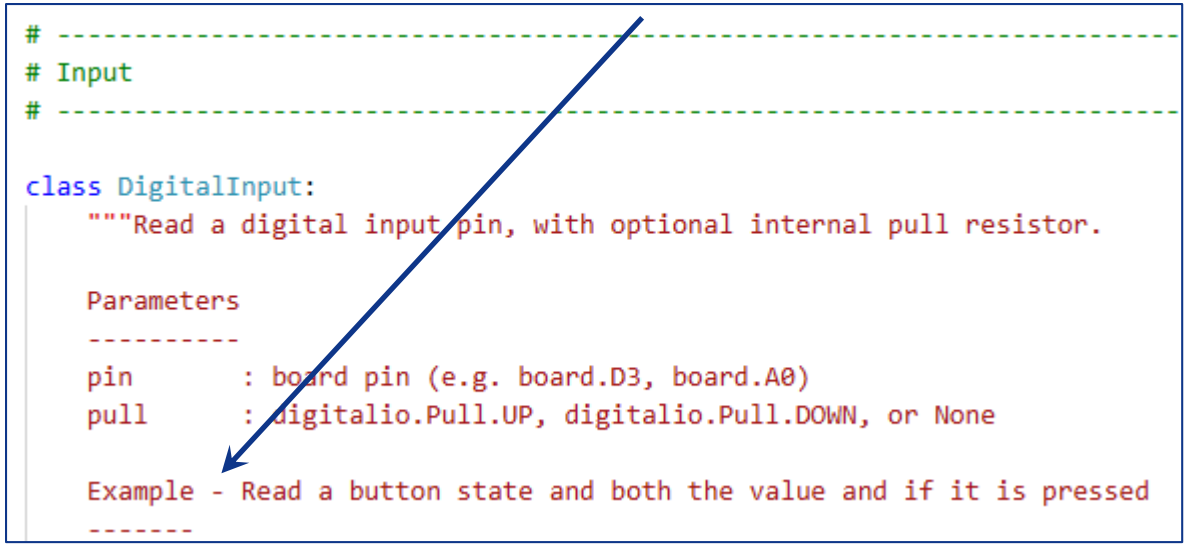
Module	Class(es)	What it does
`digital_io`	`DigitalOutput`, `DigitalInput`, `EdgeDetector`	Read buttons/switches, drive LEDs and relays; detect press/release edges

  - We want to read User Button, so we need module **digital_io**
  - Are buttons inputs or outputs?
    - Which Class(es) should we use?

Copy + Paste Examples: Read User Button

How to find the modules required?

- **Let's open the module "*digital_io*" in API folder**
- **Since our button is an INPUT, we can use Class DigitalInput**
  - There's even an example!



```
# --------------------------------------------------------------------------
# Input
# --------------------------------------------------------------------------

class DigitalInput:
    """Read a digital input pin, with optional internal pull resistor.

    Parameters
    ----------
    pin       : board pin (e.g. board.D3, board.A0)
    pull      : digitalio.Pull.UP, digitalio.Pull.DOWN, or None

    Example - Read a button state and both the value and if it is pressed
    -------
```

- **Which methods are available to this Class?**

Copy + Paste Examples: Read User Button

Methods available to Digital Input Class?

- **Constructor**

```
def __init__(self, pin, pull=digitalio.Pull.UP):
    self._pin = digitalio.DigitalInOut(pin)
    self._pin.direction = digitalio.Direction.INPUT
    if pull is not None:
        self._pin.pull = pull
    self._active_low = (pull == digitalio.Pull.UP)
```

- **Value (getter)**

```
@property
def value(self):
    """Raw pin state (True = HIGH)."""
    return self._pin.value
```

- **Is_pressed**

```
def is_pressed(self):
    """Return True when the button/switch is in its active state.

    For pull-up wiring (common): returns True when pin is LOW.
    For pull-down wiring: returns True when pin is HIGH.
    """
    return not self._pin.value if self._active_low else self._pin.value
```

- **Deinit**

```
def deinit(self):
    self._pin.deinit()
```


MICROCHIP

Copy + Paste Examples: : DigitalInput API

Reading a digital input with DigitalInput

- D3 is the user button on the dev board
  From the schematic: button is Active-LOW
  Pin reads HIGH when NOT pressed
  Pin reads LOW when pressed

- btn.value - raw pin state
  True = HIGH (not pressed)
  False = LOW (pressed)

- btn.is_pressed() - accounts for wiring
  True = button is being pressed
  False = button is released
  Uses Pull.UP internally -> inverts value

- Watch the Serial Monitor while pressing the button - what do the values show?

Minimal Example #2 — Read User Button

```
1  import pykit_explorer
2  from digital_io import DigitalInput
3
4  btn = DigitalInput(board.D3)
5
6  while True:
7      print(f'Value:      {btn.value}')
8      print(f'is pressed: {btn.is_pressed()}')
```


MICROCHIP

Project 3

Toggle LED on Button Press – Putting together Digital Inputs and Outputs

But First - What is an Edge?

The moment a signal changes state - not the state itself

- **A digital pin should always be either HIGH or LOW**

  An edge is the moment it transitions

- **Falling edge (btn.fell)**

  HIGH -> LOW transition

  Active-LOW button: means PRESSED

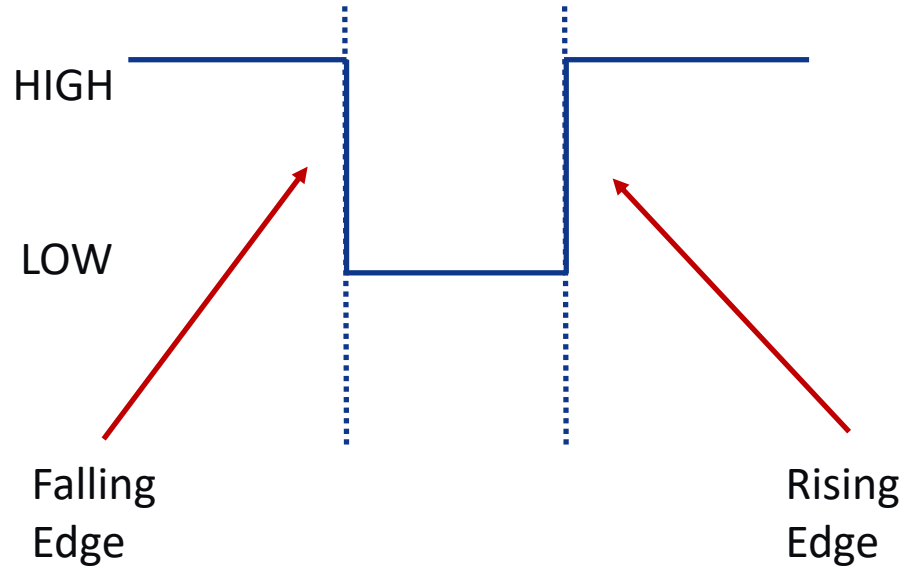
  True for exactly ONE loop iteration

- **Rising edge (btn.rose)**

  LOW -> HIGH transition

  Active-LOW button: means RELEASED

  True for exactly ONE loop iteration



But First - What is an Edge? (cont.)

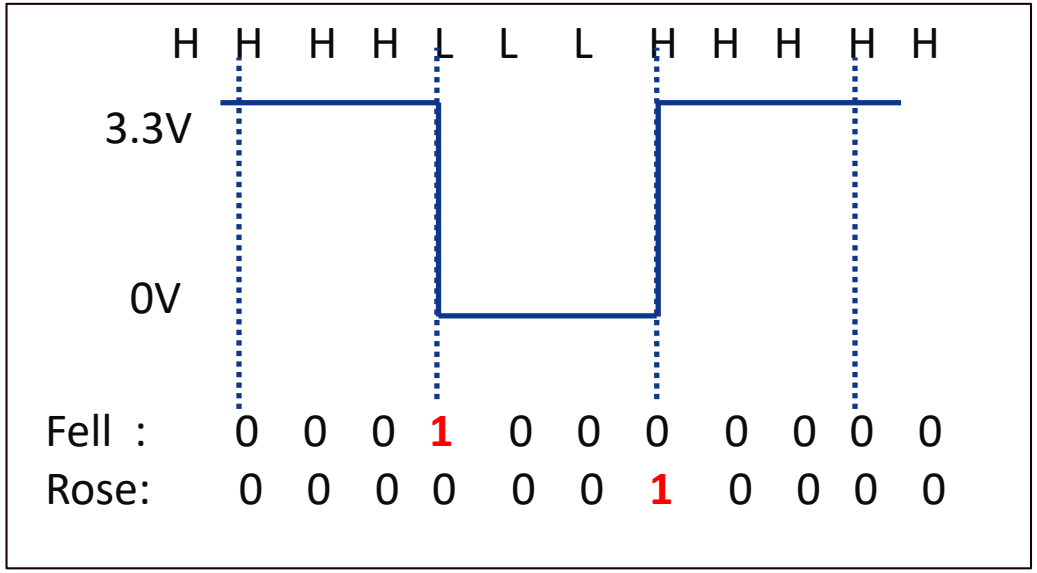
The moment a signal changes state - not the state itself

- **Why not just use btn.value?**
  btn.value = HIGH the whole time not pressed

- **The btn.value check fires every loop**
  = 100x per second at 10ms sleep!

- **btn.fell fires once at the moment**
  of press -> toggle fires exactly once

- **btn.update() must be called every loop**
  It compares current state to last state
  to detect the HIGH->LOW transition



H = High
L = Low

Project 3: Toggle LED on Button Press

EdgeDetector fires once per press - perfect for button presses!

- **import pykit_explorer**
  - Pulls in libraries: time, board, sys, builtins
  - Allows use of modules in /API folder

- **User button is connected to pin D3**
- **board.LED is the red LED near the USB port**

- **while True: runs forever**
  Without it, the code runs once and stops
- **Btn.update()**
  - Updates button state
- **If btn.fell**
  - Toggle the LED

Minimal Example #3 — Toggle LED on button press

```
1   import pykit_explorer
2   from digital_io import DigitalOutput, EdgeDetector
3
4   led = DigitalOutput(board.LED)
5   btn = EdgeDetector(board.D3)
6
7   while True:
8       btn.update()
9       if btn.fell:
10          led.toggle()
11      time.sleep(0.01)
```

Copy + Paste Examples: Toggle LED on Button Press

How to find the modules required?

- **Let's open the module "*digital_io*" in API folder**
- **Since our button is an INPUT, we can use Class EdgeDetector**
  - There's even an example!

```
class EdgeDetector:
    """Detect rising and falling edges on a digital input.

    Wraps a DigitalInput and compares the current state to the previous
    state every time ``update()`` is called.  Call ``update()`` inside
    your main loop.

    Example - Detect rising/falling edges and report events
```

- **Which methods are available to this Class?**

Copy + Paste Examples: Toggle LED on Button Press

Methods available to Edge Detector Class?

- **Constructor**

```
def __init__(self, pin, pull=digitalio.Pull.UP):
    self._input = DigitalInput(pin, pull)
    self._last = self._input.value
    self.rose = False   # True for one update cycle on LOW→HIGH transition
    self.fell = False   # True for one update cycle on HIGH→LOW transition
```

- **Update**

```
def update(self):
    """Read the pin and update rose/fell flags.  Call this every loop iteration."""
    current = self._input.value
    self.rose = (not self._last) and current
    self.fell = self._last and (not current)
    self._last = current
```

- **Value**

```
@property
def value(self):
    return self._input.value
```

- **Deinit**

```
def deinit(self):
    self._input.deinit()
```

How do we know **rose** and **fell** are flags, and NOT methods?

Project 4

Control the NeoPixels

Copy + Paste Examples: Control NeoPixels

README: Minimal Example #4

- **import pykit_explorer**
  - Pulls in libraries: time, board, sys, builtins
  - Allows use of modules in /API folder

- **Create a NeoPixels instance**

- **Fill pixels RED**
- **Set pixel #2 GREEN**
- **Color chase neopixels with BLUE**
- **Rainbow cycle**
- **Turn OFF NeoPixels**

Minimal Example #4 — NeoPixels

```
1   import pykit_explorer
2   from neopixels import NeoPixels, Colors
3
4   px = NeoPixels()  # 5 LEDs, brightness 0.1
5
6   px.fill(Colors.RED)
7   print("All Neopixels should be red")
8   time.sleep(1)
9   px.set(2, Colors.GREEN)
10  print("Pixel 2 should be green")
11  time.sleep(1)
12  px.color_chase(Colors.BLUE)
13  print("Neopixels should chase blue")
14  px.rainbow_cycle(cycles=2)
15  print("Neopixels should cycle through rainbow colors")
16  px.off()
```

Copy + Paste Examples: Control NeoPixels

How to find the modules required?

- **Open README.md**
- **Scroll down to:**
  - “Module Quick Reference”

Ruler Baseboard Modules

Module	Class(es)	What it does
`neopixels`	`NeoPixels`	Drive 5 RGB LEDs; solid colours; chase, rainbow, pulse animations; bar-graph value mapping

  - We want to control NeoPixels, so we need module **neopixels**
  - Which Class(es) should we use?

Copy + Paste Examples: Control NeoPixels

How to find the modules required?

- **Let's open the module "*neopixels*" in API folder**
- **There are 2 classes:**
  - *NeoPixels* and *Colors*
  - There's even an example!

```
class NeoPixels:
    """Drive the 5 NeoPixel LEDs on the Ruler baseboard.

    Parameters
    ----------
    pin         : NeoPixel data pin  (default board.NEOPIX)
    num_pixels  : number of LEDs     (default 5)
    brightness  : global brightness  (default 0.1, range 0.0-1.0)

    Example - Basic usage: fill, set individual pixels, and rainbow cycle
```

- **Which methods are available to this Class?**

Copy + Paste Examples: Control NeoPixels

Methods available to NeoPixels Class?

- **Constructor**

```
def __init__(self, pin=board.NEOPIXEL, num_pixels: int = 5,
             brightness: float = 0.1):
```

- **Fill**

```
def fill(self, color: tuple):
    """Set all pixels to *color* and update the strip."""
    self._pixels.fill(color)
    self._pixels.show()
```

- **Off**

```
def off(self):
    """Turn all pixels off."""
    self.fill(OFF)
```

- **Set**

```
def set(self, index: int, color: tuple, show: bool = True):
    """Set a single pixel.
```

- **Set all**

```
def set_all(self, colors: list):
    """Set each pixel from a list of (R,G,B) tuples, then update."""
```

- **Brightness**

```
@property
def brightness(self) -> float:
    return self._pixels.brightness
```

- **Color Chase**

```
def color_chase(self, color: tuple, wait: float = 0.1):
    """Light pixels one by one in *color* (blocking)."""
```

- **Rainbow Cycle**

```
def rainbow_cycle(self, wait: float = 0.0, cycles: int = 1):
    """Cycle through the colour wheel across all pixels (blocking)."""
```

- **Pulse**

```
def pulse(self, color: tuple, steps: int = 20, delay: float = 0.05):
    """Fade *color* in and out once (blocking).
```

- **Map Value**

```
def map_value(self, value: float, min_val: float = 0.0,
              max_val: float = 100.0):
    """Show a colour-mapped bar based on *value* within [min_val, max_val].
```

- **Deinit**

```
def deinit(self):
    self.off()
```

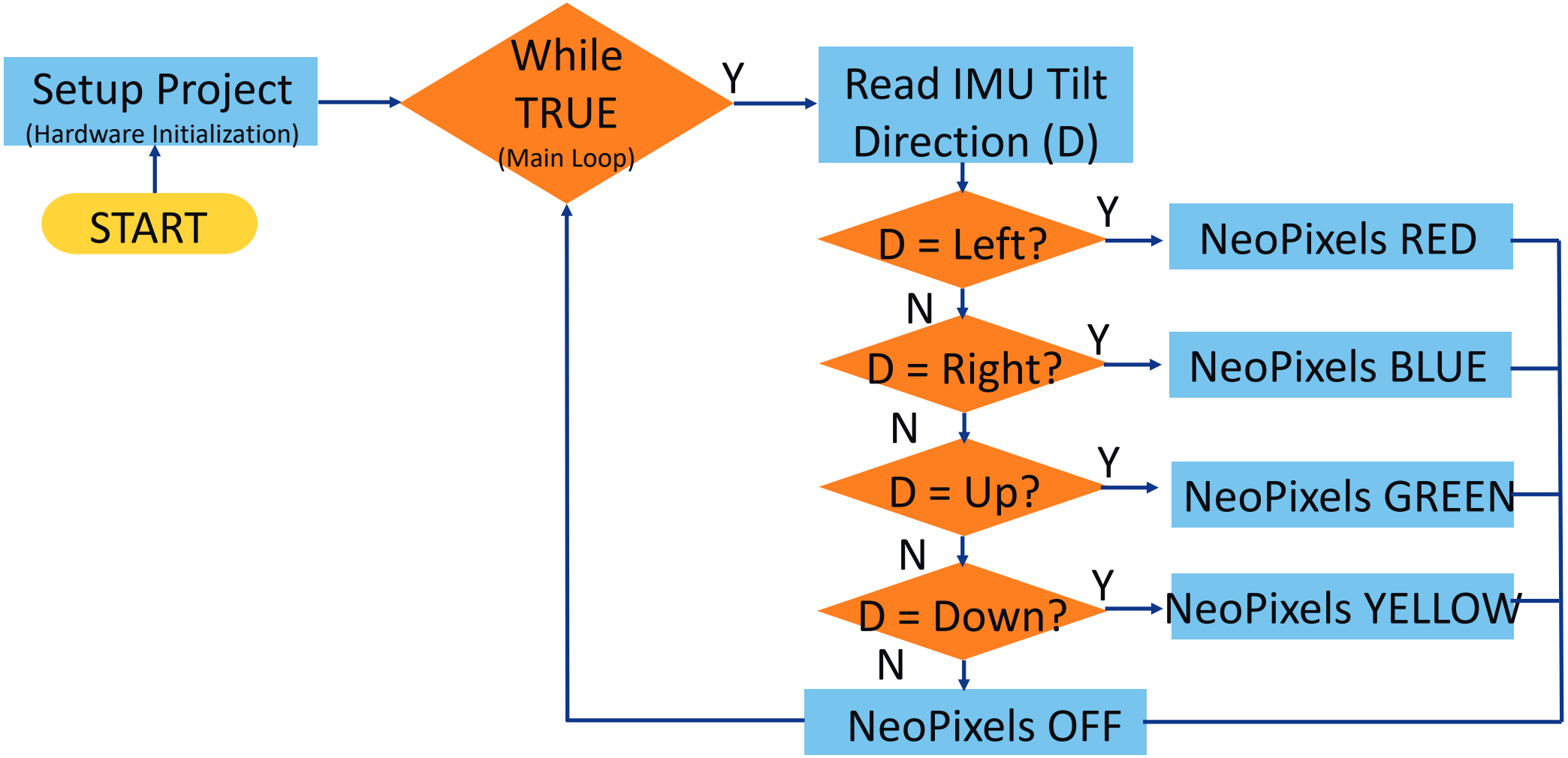

MICROCHIP

Project 5

Read IMU and Control NeoPixels

Project 5-1: Tilt-Controlled NeoPixels

Intro to Flowcharts!



MICROCHIP

Copy + Paste Examples: Tilt-Controlled NeoPixels

IMUSensor reads the ICM20948 on the Ruler

- **import pykit_explorer**
  - Pulls in libraries: time, board, sys, builtins
  - Allows use of modules in /API folder

- **Create a NeoPixels instance**
- **Create an IMUSensor instance**

- **Read direction and change NeoPixels color**

Minimal Example #5 — Tilt-controlled NeoPixel colours

```
1   import pykit_explorer
2   from imu_sensor import IMUSensor
3   from neopixels  import NeoPixels, Colors, OFF
4
5   imu = IMUSensor()
6   px  = NeoPixels()
7
8   while True:
9       direction = imu.tilt_direction()
10      if direction == "LEFT":
11          px.fill(Colors.RED)
12      elif direction == "RIGHT":
13          px.fill(Colors.BLUE)
14      elif direction == "UP":
15          px.fill(Colors.GREEN)
16      elif direction == "DOWN":
17          px.fill(Colors.YELLOW)
18      else:
19          px.off()
20
```


MICROCHIP

Copy + Paste Examples: Tilt-Controlled NeoPixels

How to find the modules required?

- **Open README.md**
- **Scroll down to:**
  - “Module Quick Reference”

Ruler Baseboard Modules

Module	Class(es)	What it does
`neopixels`	`NeoPixels`	Drive 5 RGB LEDs; solid colours; chase, rainbow, pulse animations; bar-graph value mapping
`imu_sensor`	`IMUSensor`	Read acceleration, gyro, magnetometer; tilt angles; tilt direction; sprite delta for IMU controls

- We want to control NeoPixels and read the IMU Sensor , so we need modules **neopixels** and **imu_sensor**
- Which Class(es) should we use?

Copy + Paste Examples: Tilt-Controlled NeoPixels

How to find the modules required?

- **Let's open the module "*imu_sensor*" in API folder**
- **There's only 1 class:**
  - *IMU_Sensor*
  - There's even an example!

```
class IMUSensor:
    """Read acceleration, gyroscope, and magnetometer from the ICM20948.

    Example
```

- **Which methods are available to this Class?**

Copy + Paste Examples: Tilt-Controlled NeoPixels

Methods available to IMU_Sensor Class?

- **Constructor**

```
def __init__(self, i2c=None):
```

- **Acceleration**

```
@property
def acceleration(self) -> tuple:
    """Acceleration in m/s² as (x, y, z)."""
```

- **Gyro**

```
@property
def gyro(self) -> tuple:
    """Angular velocity in rad/s as (x, y, z)."""
```

- **Magnetic**

```
@property
def magnetic(self) -> tuple:
    """Magnetic field in µT as (x, y, z)."""
```

- **Tilt angle x and y**

```
@property
def tilt_angle_x(self) -> float:
    """Board tilt about the X-axis in degrees (-90 to +90)."""
```

- **Tilt Direction**

```
def tilt_direction(self) -> str:
    """Return a coarse tilt direction string: 'LEFT', 'RIGHT', 'UP', 'DOWN', or 'FLAT'."""
```

- **Is_shaking**

```
def is_shaking(self, threshold: float = 15.0) -> bool:
    """Return True if total acceleration magnitude exceeds *threshold* m/s².
```

- **Sprite delta**

```
def sprite_delta(self, scale: float = 1.0) -> tuple:
    """Return (dx, dy) pixel deltas suitable for moving a display sprite.
```

- **Print_all**

```
def print_all(self):
    """Print all sensor axes to the console."""
```

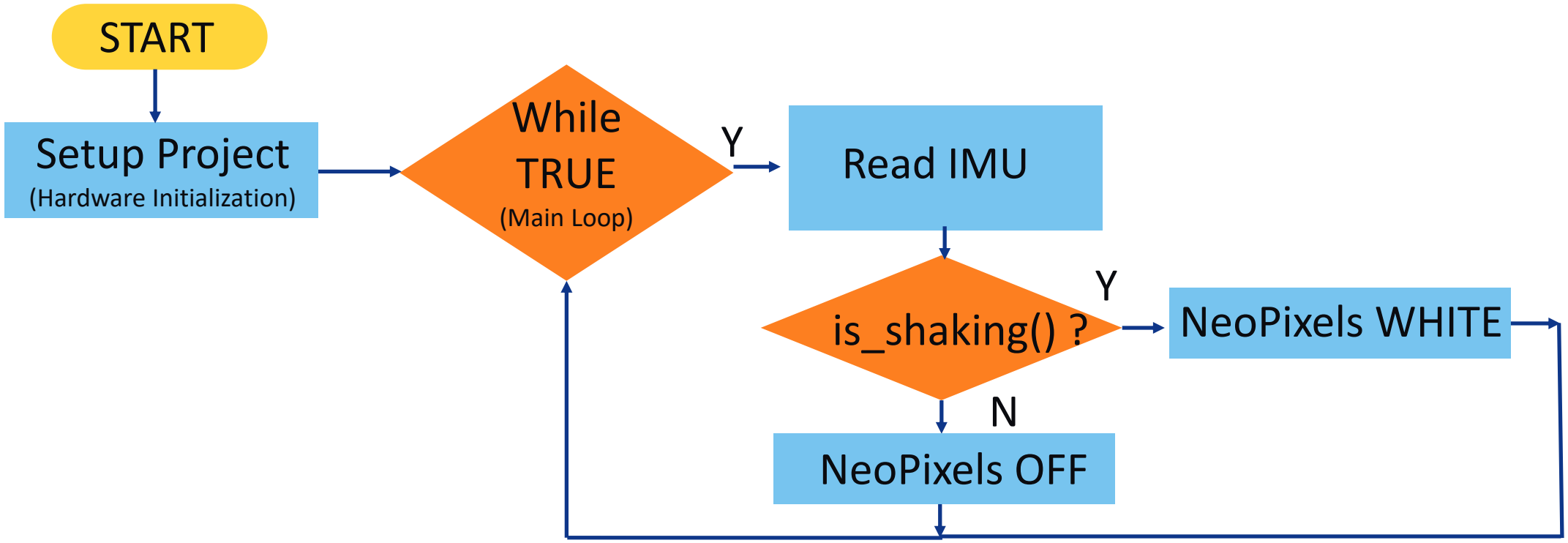
- **Log_loop**

```
def log_loop(self, interval: float = 0.2):
    """Continuously print sensor readings (blocking)."""
```


MICROCHIP

Project 5-2: IMU Shake Detection

Intro to Flowcharts!



MICROCHIP

Copy + Paste Examples: IMU Shake Detection

is_shaking() and print_all() for debugging

- **print_all() logs to Serial Monitor:**
  Accel X/Y/Z in m/s^2
  Gyro X/Y/Z in rad/s
  Mag   X/Y/Z in uT

- **is_shaking(threshold=15.0)**
  Gravity always contributes ~9.8 m/s^2
  Default 15.0 catches real shaking

Extension challenges:

- Trigger rainbow_cycle() on shake

Minimal Example #5-2 — NeoPixel & IMU shake detection

```
1   import pykit_explorer
2   from imu_sensor import IMUSensor
3   from neopixels import NeoPixels, Colors, OFF
4
5   imu = IMUSensor()
6   px  = NeoPixels()
7
8   while True:
9       imu.print_all()
10      if imu.is_shaking():
11          px.fill(Colors. WHITE)
12          time.sleep(0.2)
13      else:
14          px.off()
15      time.sleep(0.1)
```


MICROCHIP

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **IMU:**
  - a_x -> RED
  - a_y -> GREEN
  - a_z -> BLUE

- **What API modules will we need?**

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **What API modules will we need?**
  - imu_sensor
  - neopixels
- **Which classes from those modules will we need?**

MICROCHIP

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **What API modules will we need?**
  - imu_sensor
  - neopixels
- **Which classes from those modules will we need?**
  - imu_sensor -> IMUSensor
  - neopixels -> NeoPixels, Colors
- **Which methods from those classes will we need?**

MICROCHIP

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **What modules will we need?**
  - imu_sensor
  - neopixels
- **Which classes from those modules will we need?**
  - imu_sensor -> IMUSensor
  - neopixels -> NeoPixels, Colors
- **Which methods from those classes will we need?**
  - imu_sensor -> IMUSensor -> acceleration
  - neopixels -> NeoPixels -> fill
  - neopixels -> Colors -> no methods

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Import Modules and Classes**

code.py > ...

```
1   import pykit_explorer
2   #Used to read accelerometer data from the IMU sensor
3   from imu_sensor import IMUSensor
4   #Used to control the NeoPixels(LED's) and set their colors
5   from neopixels import NeoPixels, Colors
```

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Project Progression:**
- **Import required API modules and classes**
- **Instantiate an IMU object and a NeoPixels object**

MICROCHIP

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Project Progression:**
- **Import required API modules and classes**
- **Instantiate an IMU Object and a NeoPixels object**

```
#Instantiating each of the classes into Global objects
imu = IMUSensor()
px = NeoPixels()
```

MICROCHIP

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Project Progression:**
- **Import required API libraries**
- **Instantiate an IMU Object and a NeoPixels object**
- **Start by reading IMU accelerometer data**
  - What datatype does class IMUSensor, method *acceleration* return?

Complex Project – Part I

Convert IMU accelerometer data to RGB colors

- **Project Progression:**
- **Import required API libraries**
- **Instantiate an IMU Object and a NeoPixels object**
- **Start by reading IMU accelerometer data**
  - What datatype does class IMUSensor, method *acceleration* return?
    - A tuple – but a tuple of what?

```
class IMUSensor:
@property
def acceleration(self) -> tuple:
    """Acceleration in m/s² as (x, y, z)."""
    return self._icm.acceleration
```

    - How many objects in the tuple, what datatype/s are they?

MICROCHIP

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Project Progression:**
- **Import required API libraries**
- **Instantiate an IMU Object and a NeoPixels object**
- **Start by reading IMU accelerometer data**
  - What datatype does class IMUSensor, method *acceleration* return?
    - A tuple – but a tuple of what?
    - Acceleration is measured in m/s^2
    - E.g., Gravity is acceleration = 9.8 m/s^2 **Datatype is float**
    - IMU measures acceleration in 3-axes (x, y, z)
    - How many objects in the tuple, what datatype/s are they? **3 objects of type float**
    - **We can print to Serial Monitor to confirm datatypes and number of items in tuple!**

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

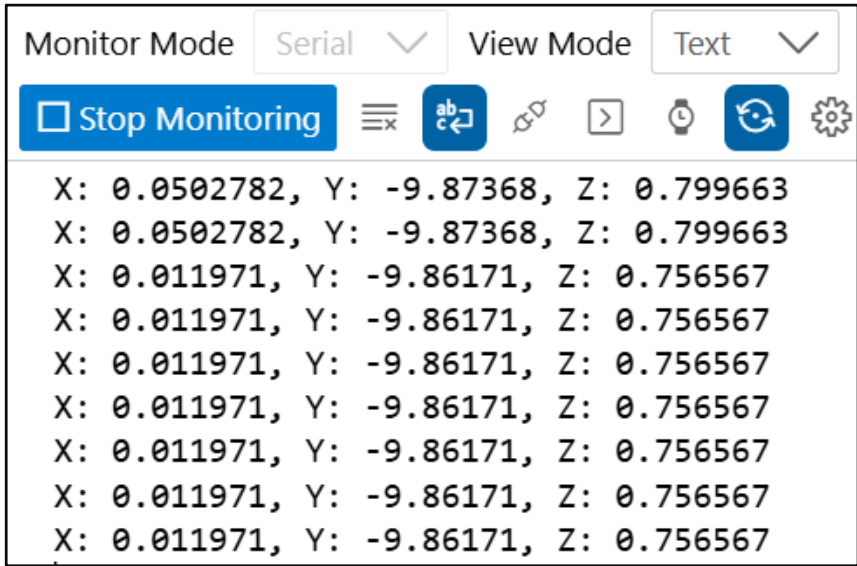
- **Project Progression:**
- **Import required API modules and classes** 
- **Instantiate an IMU Object and a NeoPixels object** 
- **Start by reading IMU accelerometer data and print to Serial Monitor**

```
while True:
    # Gets the acceleration values from the IMU sensor and assign
    # to the variables ax, ay, az for x, y, z axis
    ax, ay, az = imu.acceleration
    print(f"X: {ax}, Y: {ay}, Z: {az}")
```

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- Project Progression:
- Import required API modules and classes
- Instantiate an IMU Object and a NeoPixels object
- Start by reading IMU accelerometer data
- Serial Monitor output
- Move the PyKit around to see (x,y,z) range



Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Project Progression:**
- **Import required API libraries**
- **Instantiate an IMU Object and a NeoPixels Object**
- **Read IMU data and print to Serial Monitor**
- **Move the PyKit around to see x,y,z range**
- **Map x,y,z output to RGB input**
  - What input does NeoPixel object, Fill method take?
  - How many objects in the tuple?

```
color : (R, G, B) tuple
```

MICROCHIP

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Project Progression:**
- **Import required API libraries**
- **Instantiate an IMU Object and a NeoPixels Object**
- **Read IMU data and print to Serial Monitor**
- **Move the PyKit around to see x,y,z range**
- **Map x,y,z output to RGB input**
  - What input does NeoPixel object, Fill method take? **A Tuple**
  - How many objects in the tuple? **Three**
  - What datatype/s in the tuple?

```
color : (R, G, B) tuple
```


MICROCHIP

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Project Progression:**
- **Import required API libraries**
- **Instantiate an IMU Object and a NeoPixels Object**
- **Read IMU data and print to Serial Monitor**
- **Move the PyKit around to see x,y,z range**
- **Map x,y,z output to RGB input**
  - What datatype/s in the tuple?

```
34    class Colors:
35
36        WHITE       = 0xFFFFFF
37        RED         = 0xFF0000
38        GREEN       = 0x00FF00
39        BLUE        = 0x0000FF
40        ORANGE      = 0xFF8000
41        YELLOW      = 0xFFFF00
42        CYAN        = 0x00FFFF
43        PURPLE      = 0xB400FF
```

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Create NeoPixel Object**
  - What datatype does NeoPixel object tuple take?

```
34    class Colors:
35
36        WHITE          = 0xFFFFFF
37        RED            = 0xFF0000
38        GREEN          = 0x00FF00
39        BLUE           = 0x0000FF
40        ORANGE         = 0xFF8000
41        YELLOW         = 0xFFFF00
42        CYAN           = 0x00FFFF
43        PURPLE         = 0xB400FF
```

Range for each of RED, GREEN, BLUE:

0x00 to 0xFF (hexadecimal)

What's that as a decimal?

How many bits is that?

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Create NeoPixel Object**
  - What datatype does NeoPixel object take?

```
34    class Colors:
35
36        WHITE          = 0xFFFFFF
37        RED            = 0xFF0000
38        GREEN          = 0x00FF00
39        BLUE           = 0x0000FF
40        ORANGE         = 0xFF8000
41        YELLOW         = 0xFFFF00
42        CYAN           = 0x00FFFF
43        PURPLE         = 0xB400FF
```

Range for each of RED, GREEN, BLUE:

0x00 to 0xFF (hexadecimal)

0 to 255 (decimal)

Each of R, G, B is an 8-bit value

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

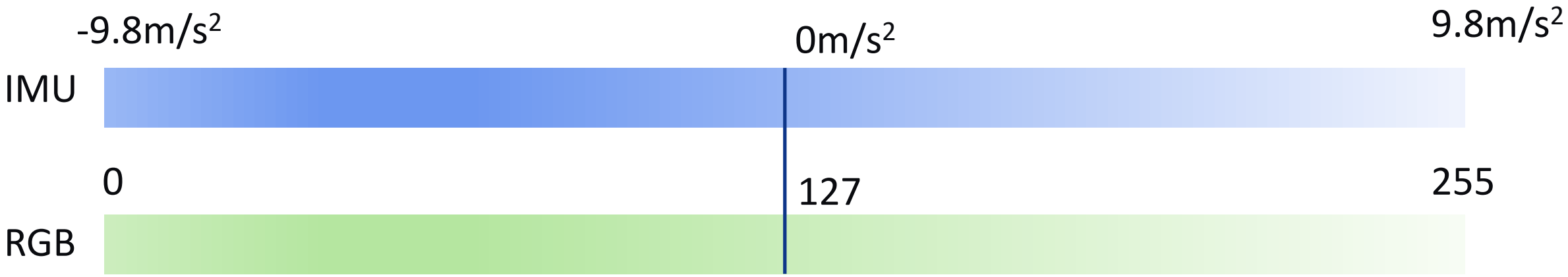
- **Project Progression:**
- **Import required API modules and classes**
- **Instantiate an IMU Object and a NeoPixels object**
- **Start by reading IMU accelerometer data**
- **Serial Monitor output**
- **Move the PyKit around to see (x,y,z) range**
- **Convert signed float(x,y,z) -> unsigned 8-bit int (R,G,B)**
  - Remember accelerometer values can be negative or positive!

MICROCHIP

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

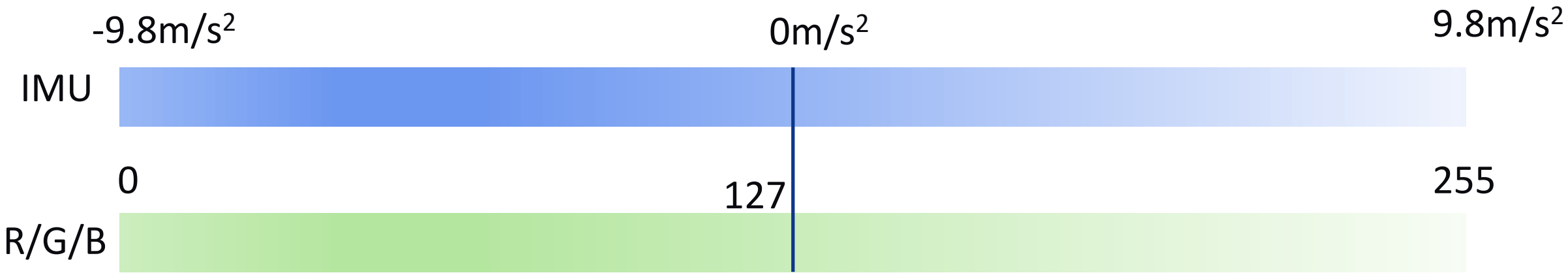
- **We need a way of mapping:**
  - IMU readings (float) → RGB inputs (integer)
- **But wait, there's more!**
  - IMU values range ±1g (-9.8m/s² to 9.8m/s²)
  - RGB inputs range from 0 to 255



Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- $\boldsymbol{RGB_{Output}} = \left(\frac{\boldsymbol{accel+Gravity}}{(\boldsymbol{2})\ (\boldsymbol{Gravity})}\right) * \boldsymbol{RGB_{MAX}}$
- This centers the mapping so that 0g acceleration produces 127 (mid-range RGB)
-



Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **First, we need to define some constants:**
  - For simplicity we can round up gravity from ±9.8m/s^2 to ±10m/s^2
  - Minimum input for RGB is 0 (0x00)
  - Maximum value for RGB is 255 (0xFF)

```
#Constants
GRAVITY = 10
RGB_MAX = 255
RGB_MIN = 0
```

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Next, we define a method to implement the equation:**

```
def acceleration_to_rgb(ax, ay, az):
    #maps the acceleration value to a range of 0-255 for red
    r = int(((ax+GRAVITY)/(GRAVITY*2))*RGB_MAX)
    #maps the acceleration value to a range of 0-255 for green
    g = int(((ay+GRAVITY)/(GRAVITY*2))*RGB_MAX)
    #maps the acceleration value to a range of 0-255 for blue
    b = int(((az+GRAVITY)/(GRAVITY*2))*RGB_MAX)
```

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **We should clamp the values, in case the user shakes the ruler:**

```
if   (r > RGB_MAX):    r = RGB_MAX
elif(r < RGB_MIN):    r = RGB_MIN
if   (g > RGB_MAX):    g = RGB_MAX
elif(g < RGB_MIN):    g = RGB_MIN
if   (b > RGB_MAX):    b = RGB_MAX
elif(b < RGB_MIN):    b = RGB_MIN
```

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Method must be defined before *while True:***

```
def acceleration_to_rgb(ax, ay, az):
    #maps the acceleration value to a range of 0-255 for red
    r = int(((ax+GRAVITY)/(GRAVITY*2))*RGB_MAX)
    #maps the acceleration value to a range of 0-255 for green
    g = int(((ay+GRAVITY)/(GRAVITY*2))*RGB_MAX)
    #maps the acceleration value to a range of 0-255 for blue
    b = int(((az+GRAVITY)/(GRAVITY*2))*RGB_MAX)
    if  (r > RGB_MAX):    r = RGB_MAX
    elif(r < RGB_MIN):    r = RGB_MIN
    if  (g > RGB_MAX):    g = RGB_MAX
    elif(g < RGB_MIN):    g = RGB_MIN
    if  (b > RGB_MAX):    b = RGB_MAX
    elif(b < RGB_MIN):    b = RGB_MIN
    # Return the RGB values as a tuple to be used for NeoPixel color
    return (r, g, b)
```

Complex Project – Part I

Task: Convert IMU accelerometer data to RGB colors on the NeoPixels

- **Then in *while True:***

```
while True:
    # Gets the acceleration values from the IMU sensor
    ax, ay, az = imu.acceleration
    print(f"X: {ax}, Y: {ay}, Z: {az}")
    # Convert the acceleration values to RGB values
    r, g, b = acceleration_to_rgb(ax, ay, az)
    px.fill((r, g, b))  # Pass tuple to NeoPixels
```

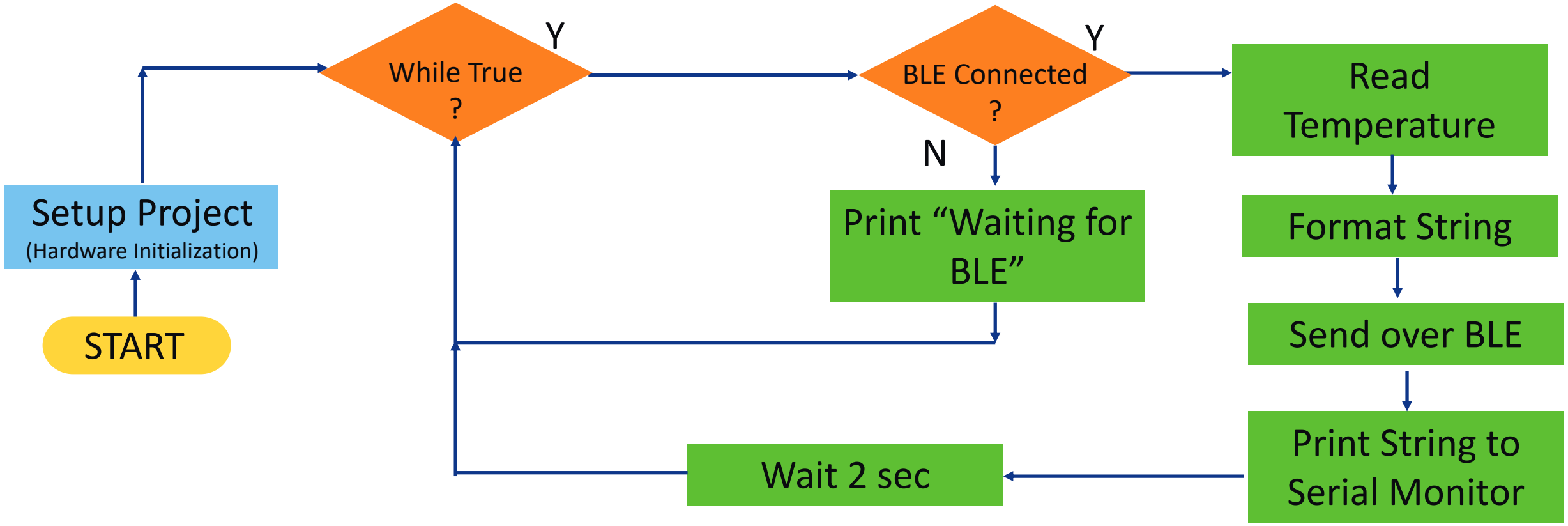
Project 6

BLE Temperature Logger

MICROCHIP

Project 6: BLE Temperature Logger

Send CPU temperature wirelessly to your phone



MICROCHIP

Project 6: BLE Temperature Logger

First Install the Microchip Bluetooth Data app!

Android

iPhone

Copy + Paste Examples: BLE Temperature Logger

Send CPU temperature wirelessly to your phone

- **import pykit_explorer**
  - Pulls in libraries: time, board, sys, builtins
  - Allows use of modules in /API folder

- **Create a BLEUart instance**
- **Create an CPUTemperature instance**

- **Update BLE connection state (poll)**
- **If BLE is connected:**
  - Send CPU Temperature
  - Print CPU Temperature to Serial Monitor

Minimal Example #6-1 — BLE temperature logger

```
1   import pykit_explorer
2   from ble_uart import BLEUart
3   from cpu_temp import CPUTemperature
4
5   ble  = BLEUart()
6   temp = CPUTemperature()
7
8   while True:
9       ble.poll()  # process connection status messages
10      if ble.connected:
11          ble.send(f"Temp: {temp.formatted_string()}\n")
12          print(f"Temp: {temp.formatted_string()}")
13      time.sleep(2)
```


MICROCHIP

Copy + Paste Examples: BLE Temperature Logger

How to find the modules required?

- **Open README.md**
- **Scroll down to:**
  - “Module Quick Reference”

Module Quick Reference

Dev Board Modules

Module	Class(es)	What it does
`cpu_temp`	`CPUTemperature`	On-chip temperature in °C and °F; threshold checks; formatted logging strings
`ble_uart`	`BLEUart`	Reset RNBD451 BLE module; send/receive strings wirelessly; connection status tracking

- We want to read CPU Temperature and send over BLE UART, so we need modules **cpu_temp** and **ble_uart**
- Which Class(es) should we use?

MICROCHIP

Copy + Paste Examples: BLE Temperature Logger

Methods available to BLEUart?

- **Constructor**

```
def __init__(self, baudrate: int = 115200):
```

- **Connected**

```
@property
def connected(self) -> bool:
    """True if a BLE central is connected (stack or hardware proto events)."""
```

- **Just_connected**

```
@property
def just_connected(self) -> bool:
    """True during the first poll() call after a connection is established."""
```

- **Just_disconnected**

```
@property
def just_disconnected(self) -> bool:
    """True during the first poll() call after disconnection."""
```

- **Poll**

```
def poll(self) -> str:
    """Read BLE data and update connection state. Call once per main loop.
```

- **Send**

```
def send(self, text: str, encoding: str = "ascii"):
    """Send a string over BLE to the connected central device."""
```

- **Send_bytes**

```
def send_bytes(self, data: bytes):
    """Send raw bytes over BLE."""
```

- **Receive**

```
def receive(self, num_bytes: int = 64, debug: bool = False) -> str:
    """Low-level receive — returns raw user data with %STATUS% stripped.
```

- **Deinit**

```
def deinit(self):
```


MICROCHIP

Copy + Paste Examples: BLE Temperature Logger

Methods available to CPUTemperature?

- **Constructor – None!**

- **Celsius**

```
@property
def celsius(self) -> float:
    """CPU temperature in degrees Celsius."""
```

- **Fahrenheit**

```
@property
def fahrenheit(self) -> float:
    """CPU temperature in degrees Fahrenheit."""
```

- **Log_once**

```
def log_once(self):
    """Print a single temperature reading to the console."""
```

- **Log_loop**

```
def log_loop(self, interval: float = 1.0):
    """Continuously print temperature every *interval* seconds (blocking)."""
```

- **Is_above**

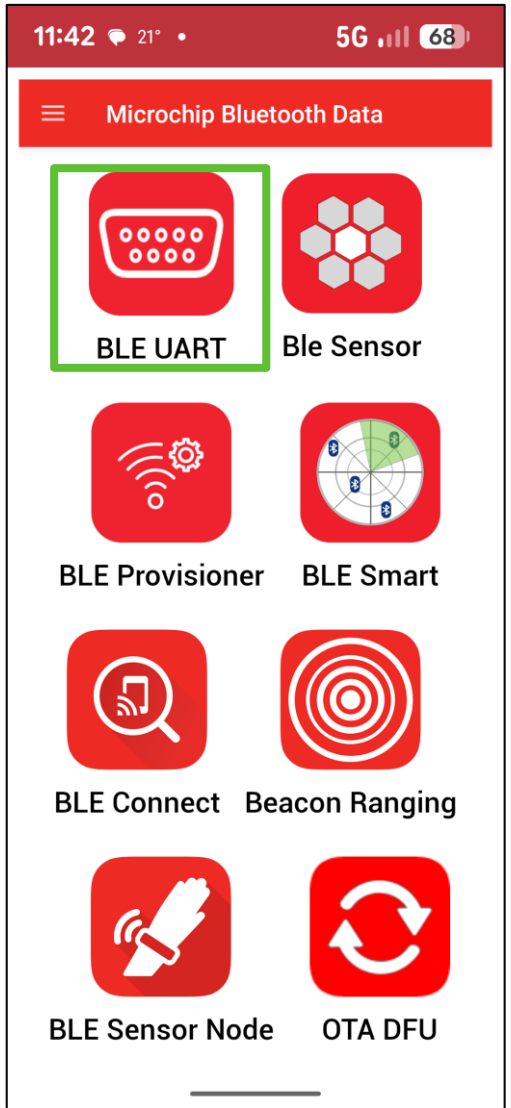
```
def is_above(self, threshold_c: float) -> bool:
    """Return True if the CPU temperature exceeds *threshold_c* °C."""
```

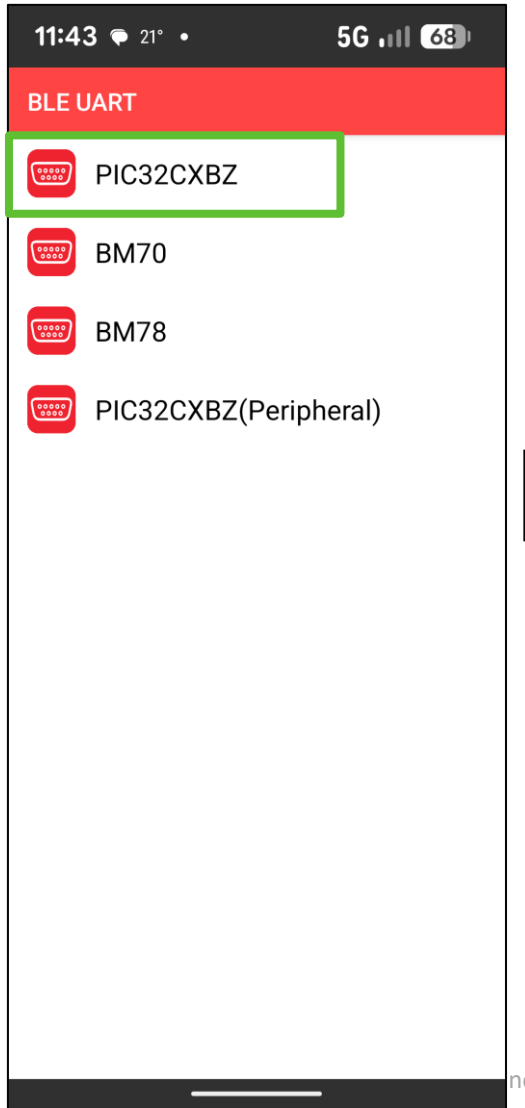
- **Formatted_string**

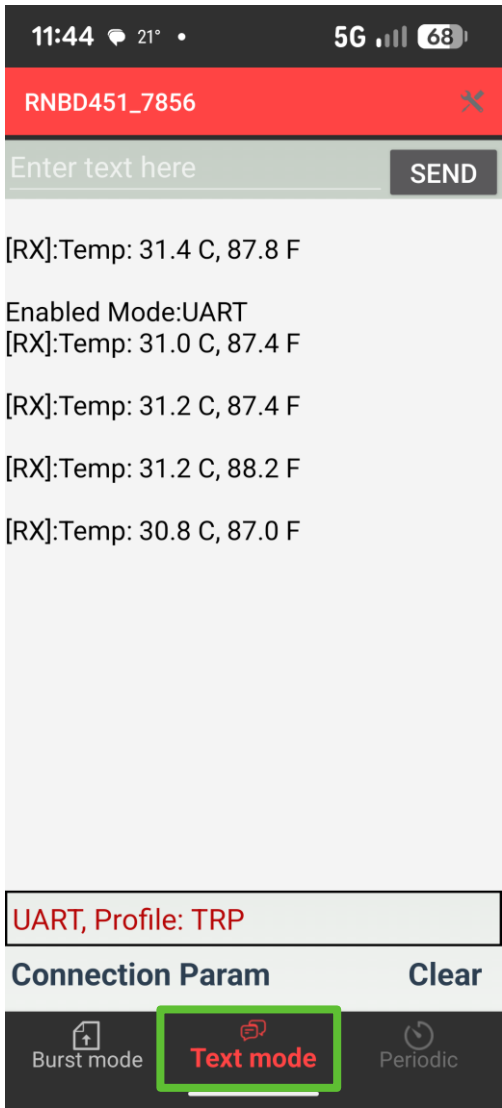
```
def formatted_string(self) -> str:
    """Return a formatted string suitable for UART or SD card logging."""
```


MICROCHIP

How to Use the App







Project 6: Receiving BLE Commands

Two-way BLE: control NeoPixels from phone

- **import pykit_explorer**
  - Pulls in libraries: time, board, sys, builtins
  - Allows use of modules in /API folder

- **Create a BLEUart instance**
- **Create an NeoPixels instance**

Send commands in the Microchip Bluetooth Data app

BLE receives command (cmd) and acts on them:

- **Always call .strip() to remove newlines (“/n”)**
- **Use repr(cmd) to debug whitespace**

Minimal Example #6-2 — Receiving BLE Commands

```
1   import pykit_explorer
2   from ble_uart import BLEUart
3   from neopixels import NeoPixels, Colors, OFF
4
5   ble  = BLEUart()
6   px   = NeoPixels()
7
8   while True:
9       cmd = ble.receive().strip()
10      if cmd == 'RED': px.fill(Colors.RED)
11      elif cmd == 'GREEN': px.fill(Colors.GREEN)
12      elif cmd == 'OFF': px.off()
13      if cmd:
14              print(f'Got: {repr(cmd)}')
15      time.sleep(0.05)
```


MICROCHIP

Complex Project - Part 2

Adding BLE Support to Our Project

- **Requirements:**
  - Read IMU data
  - Make NeoPixels change color using IMU data
  - Write IMU data & NeoPixel colors to BLE

Questions:

- **What modules do we need?**
- **Which classes do we need?**
- **Which methods do we need?**
- **What datatype/s does:**
  - LCD labels take?
  - BLE send take?

Complex Project Part 2

Adding BLE Support to Our Project

- **Import:**
  - ble_uart classes

```
from ble_uart import BLEUart
```

- **Instantiate ble object**

```
ble = BLEUart()
```

- **Inside while True: loop**
  - Put BLE code

```
#sending data to the app via bluetooth
msg = ble.poll()
if ble.connected:
    ble.send("ax: " + str(round(ax, 2))+"\n" +
             "ay: " + str(round(ay, 2))+"\n" +
             "az: " + str(round(az, 2))+"\n" +
             "RGB: " + str(rgbcoordinate)+ "\n\n")
```

Project 7

Write to LCD Using Colored Labels

Copy + Paste Examples: Write to LCD Using Labels

Create Name Labels and Value Labels and Update Them

- **Import pykit_explorer**
- **Import random**
  - To generate random numbers
- **Import LCDDisplay class and Colors class from lcd_display module**
- **Instantiate lcd object**
  - Turn backlight on
- **Create an LCD group**
  - Group = Background
  - Set to BLACK
- **Create list for:**
  - Label colors
  - Label names
  - Y-positions (coordinates on LCD)

```
import pykit_explorer
import random
from lcd_display import LCDDisplay, Colors

lcd = LCDDisplay()
lcd.backlight_on()

group, _ = lcd.make_group(Colors.BLACK)

LABEL_COLORS = [Colors.RED, Colors.GREEN, Colors.BLUE, Colors.WHITE]
LABEL_NAMES  = ["Label 1", "Label 2", "Label 3", "Label 4"]
Y_POSITIONS  = [20, 50, 80, 110]
```

Copy + Paste Examples: Write to LCD Using Labels

Create Name Labels and Value Labels and Update Them (cont.)

- **Create an empty list for name labels**
- **Create an empty list for value labels**
- **Iterate (for-loop):**
  - Add name label
  - Add value label
  - Append name label to group
  - Append value label to group
- **While True (loop)**
  - Iterate (for loop)
    - Update value labels with random values
  - Wait 0.5 sec

```
name_labels  = []
value_labels = []

for i in range(4):
    name_lbl = lcd.add_label(group, LABEL_NAMES[i], 60, Y_POSITIONS[i],
                             color=LABEL_COLORS[i], scale=2)
    value_lbl = lcd.add_label(group, "0.00", 180, Y_POSITIONS[i],
                              color=LABEL_COLORS[i], scale=2)
    name_labels.append(name_lbl)
    value_labels.append(value_lbl)

while True:
    for i in range(4):
        value_labels[i].text = f"{random.uniform(0, 100):.2f}"
    time.sleep(0.5)
```

Copy + Paste Examples: Write to LCD Using Labels

Methods available to LCDDisplay?

- **Constructor**

```
def __init__(self):
```

- **Turn backlight on**

```
def backlight_on(self):
    """Turn the LCD backlight on."""
```

- **Turn backlight off**

```
def backlight_off(self):
    """Turn the LCD backlight off."""
```

- **Make group**

```
def make_group(self, bg_color: int = Colors.BLACK):
    """Create a full-screen displayio Group with a solid background.
```

- **Add label**

```
def add_label(self, group: displayio.Group, text: str,
              x: int, y: int,
              color: int = 0xFFFFFF,
              scale: int = 2) -> "_label.Label":
    """Create a horizontally-centred label and append it to *group*.
```

- **Scroll label**

```
def scroll_label(self, label, text: str, y: int,
                 scale: int = 2, step: int = 4, delay: float = 0.0,
                 poll=None):
    """Scroll text across the full display width, right to left.
```

- **Fill screen**

```
def fill_screen(self, color_565: int = 0x0000):
    """Fill the entire screen with a 16-bit RGB565 colour.
```

- **Load sprite**

```
def load_sprite(self, bmp_path: str, sprite_w: int = WIDTH, sprite_h: int = HEIGHT,
                x: int = 0, y: int = 0) -> displayio.Group:
    """Load a BMP sprite sheet and return a positioned displayio.Group.
```

- **And too many more to fit here!**

MICROCHIP

Complex Project - Part 3

Add LCD Header

- **Requirements:**
  - Read IMU data 
  - Make NeoPixels change color  using IMU data
  - Write IMU data & NeoPixel colors to BLE 
  - Write Header to LCD

Questions:

- **What modules do we need?**
- **Which classes do we need?**
- **Which methods do we need?**
- **What datatype/s does:**
  - LCD labels take?

Complex Project - Part 3

Add LCD Header

Heading

MICROCHIP

Complex Project - Part 3

Add LCD Header

- **Import:**
  - LCDDisplay class

```
from lcd_display import LCDDisplay, Colors
```

- **Instantiate lcd object**

```
lcd = LCDDisplay()
```

- **Create a “group” for the display**

```
# Create a "container" (called a group) for the display
# elements on the LCD, and set the background color to black.
group, palette = lcd.make_group(Colors.BLACK)
```

- **Add the header label to the LCD**

```
#Header label for the project title
imuLabel0 = lcd.add_label(group, "IMU to RGB project ",
                          120, 0, scale=2, color=Colors.GREEN)
```

Complex Project - Part 4

Add IMU Labels for “ax”, “ay”, “az”

- **Requirements:**
  - Read IMU data
  - Make NeoPixels change color using IMU data
  - Write IMU data & NeoPixel colors to BLE
  - Write Header to LCD
  - Write IMU name labels (ax, ay, az) to the LCD

Questions:

- **Which methods do we need?**
- **What datatype/s does:**
  - LCD labels take?

Complex Project - Part 4

Add IMU name labels

Heading

ax, ay, az labels

Complex Project Part 4

Add IMU Labels to LCD

- **Create labels for IMU data**

```
# IMU acceleration labels (name + value pairs)
# for ax, ay, az.
IMU_NAMES       = ["ax: ", "ay: ", "az: "]
IMU_Y_POSITIONS = [25, 45, 65]
```

- **Create empty lists for labels to append to**

```
imu_name_labels  = []
imu_value_labels = []
```

- **Append each label to the group**

```
# Add then append each label to the group and the corresponding list for later access
for i in range(3):
    name_lbl = lcd.add_label(group, IMU_NAMES[i], 40, IMU_Y_POSITIONS[i], scale=2)
    value_lbl = lcd.add_label(group, "", 100, IMU_Y_POSITIONS[i], scale=2)
    imu_name_labels.append(name_lbl)
    imu_value_labels.append(value_lbl)
```

Complex Project Part 5

Add IMU data to LCD

- **Requirements:**
  - Read IMU data
  - Make NeoPixels change color using IMU data
  - Write IMU data & NeoPixel colors to BLE
  - Write Header to LCD
  - Write IMU name labels (ax, ay, az) to the LCD
  - Add IMU data to LCD

Questions:

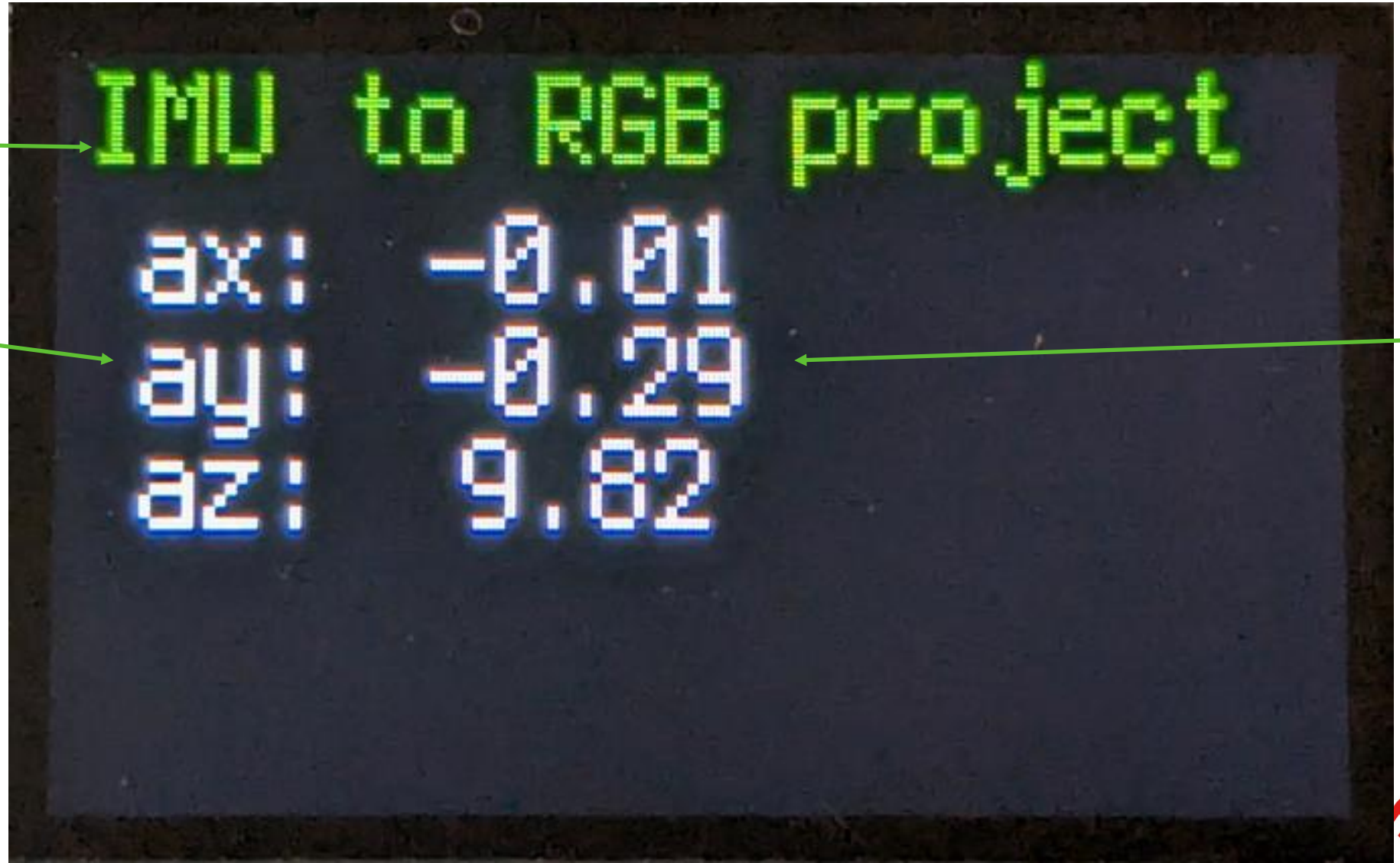
- **Which methods do we need?**
- **What datatype/s does:**
  - LCD labels take?

Complex Project - Part 5

Add IMU data labels

Heading

IMU Labels



IMU Data

Complex Project – Part 5

Update IMU value labels

- **Write IMU data to LCD**

```
#displaying IMU data on the lcd
# ax, ay, az values rounded to 2 decimal places
imu_value_labels[0].text = str(round(ax, 2))
imu_value_labels[1].text = str(round(ay, 2))
imu_value_labels[2].text = str(round(az, 2))
```

Complex Project Part 6

Add RGB labels to LCD

- **Requirements:**
  - Read IMU data
  - Make NeoPixels change color using IMU data
  - Write IMU data & NeoPixel colors to BLE
  - Write Header to LCD
  - Write IMU name labels (ax, ay, az) to the LCD
  - Add IMU data to LCD
  - Write RGB name labels to the LCD

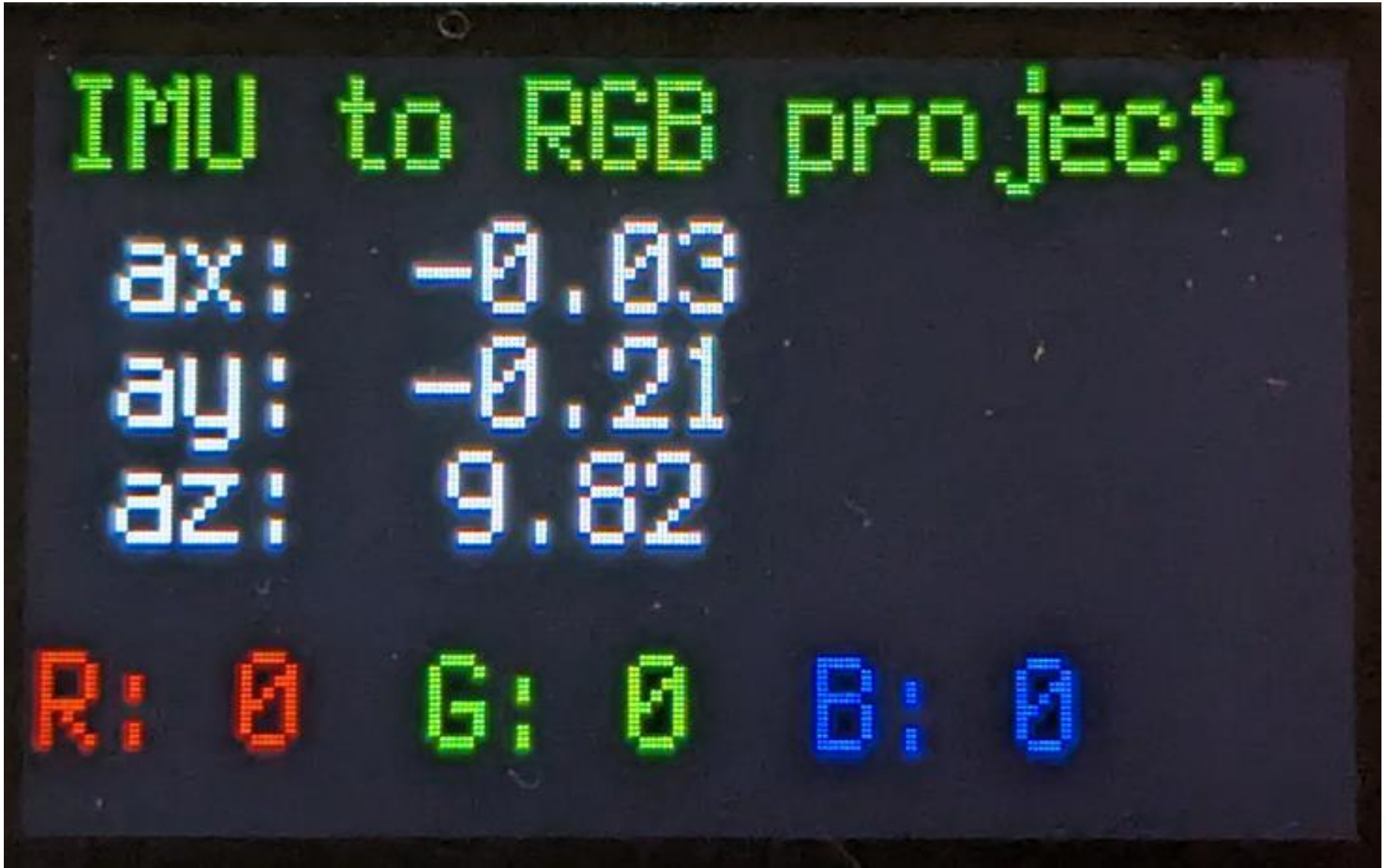
Questions:

- **Which methods do we need?**
- **What datatype/s does:**
  - LCD labels take?

Complex Project – Part 6

Add RGB labels to LCD

RGB Labels



Complex Project – Part 6

Add RGB Labels to LCD

- **Create labels for RGB data**

```
# RGB value labels (name + value pairs) for R, G, B.
RGB_NAMES              = ["R:", "G:", "B:"]
RGB_COLORS             = [Colors.RED, Colors.GREEN, Colors.BLUE]
RGB_NAME_X_POSITIONS   = [15, 85, 155]
RGB_VALUE_X_POSITIONS = [45, 115, 185]
```

- **Create empty lists for labels to append to**
- **Append each label to the group**

```
rgb_name_labels  = []
rgb_value_labels = []
```

```
for i in range(3):
    name_lbl = lcd.add_label(group, RGB_NAMES[i], RGB_NAME_X_POSITIONS[i], 100,
                             scale=2, color=RGB_COLORS[i])
    value_lbl = lcd.add_label(group, "0", RGB_VALUE_X_POSITIONS[i], 100,
                              scale=2, color=RGB_COLORS[i])
    rgb_name_labels.append(name_lbl)
    rgb_value_labels.append(value_lbl)
```

Complex Project Part 7

Write RGB data to LCD

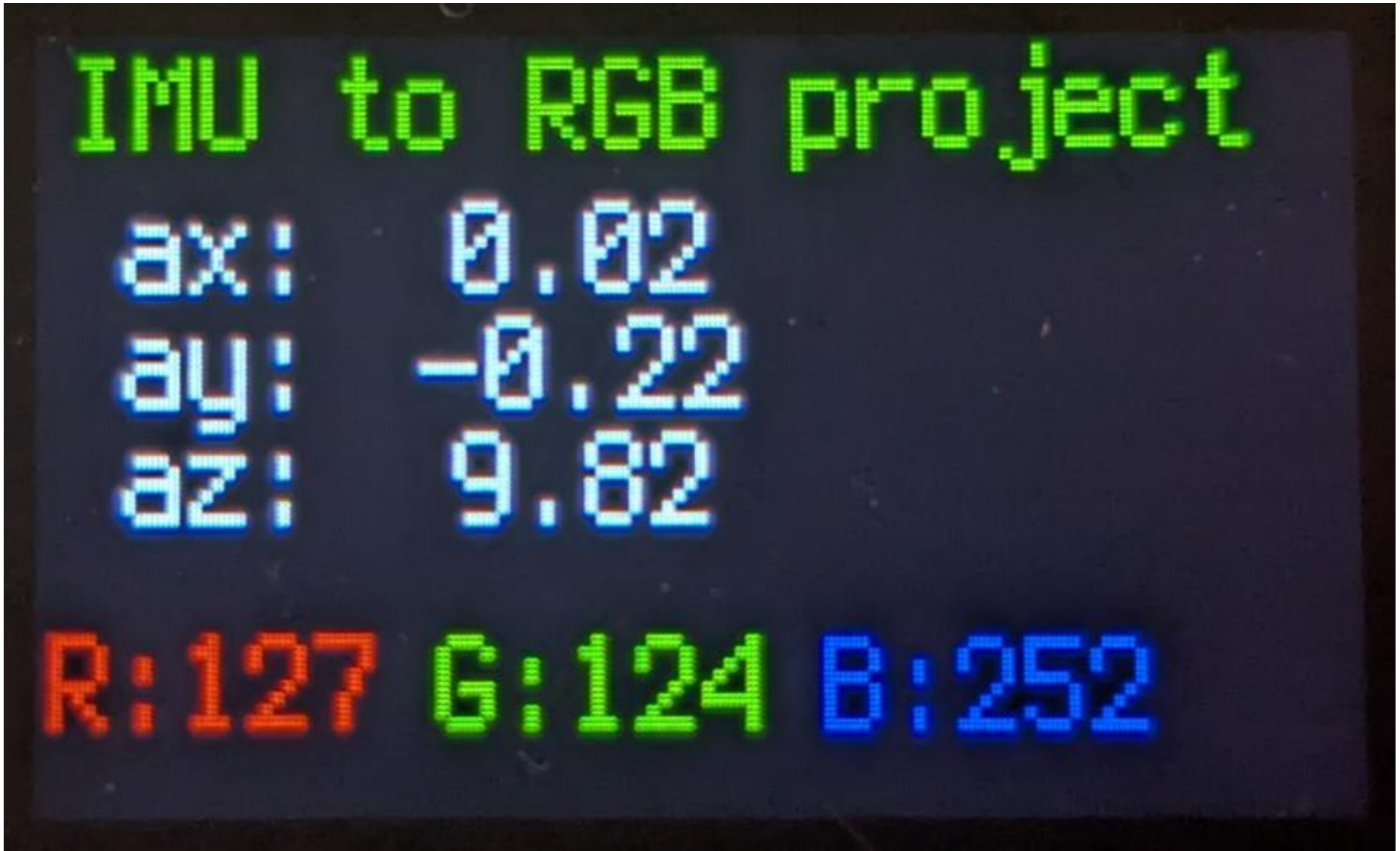
- **Requirements:**
  - Read IMU data
  - Make NeoPixels change color using IMU data
  - Write IMU data & NeoPixel colors to BLE
  - Write Header to LCD
  - Write IMU name labels (ax, ay, az) to the LCD
  - Add IMU data to LCD
  - Write RGB name labels to the LCD
  - Write RGB data to the LCD

Questions:

- **Which methods do we need?**
- **What datatype/s does:**
  - LCD labels take?

Complex Project Part 7

Write RGB data to LCD



MICROCHIP

Complex Project – Part 7

Write RGB Data to LCD

- **In while True: loop**

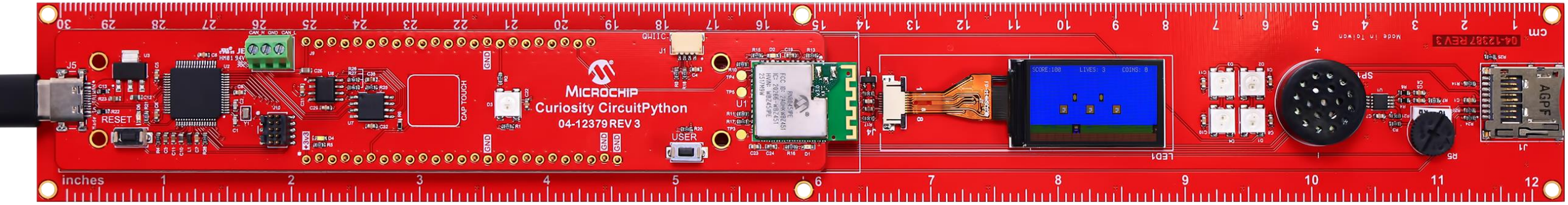
```
# r,g,b values displayed on the LCD
rgb_value_labels[0].text = str(r)
rgb_value_labels[1].text = str(g)
rgb_value_labels[2].text = str(b)
```

- **Don’t forget to save your code!**

Congratulations!

You Have Completed The Student Workshop!

- **What do you want to build next?**



MICROCHIP